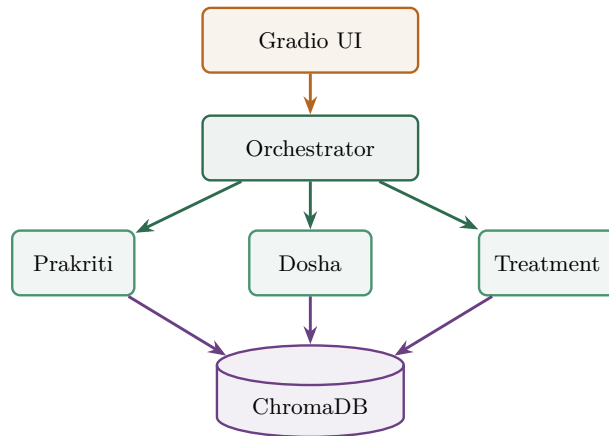

AyurMind

Codebase Documentation

A multi-agent retrieval system over the Charaka Samhita



Repository: [AyurMind](#)

Documented from commit 9213703 on branch `main`

26 July 2026

Contents

1	What AyurMind Is	1
1.1	The problem it targets	1
1.2	Who the output is written for	1
1.3	The two paths through the system	1
1.4	Technology in one table	2
2	Repository Layout	3
2.1	Directories the code creates but git ignores	3
2.2	Files the README promises that do not exist	4
2.3	Reading order for a newcomer	4
3	Getting It Running	5
3.1	What you need	5
3.2	Install	5
3.3	Configuration	5
3.4	The four-step pipeline	6
3.4.1	What step 1 actually prints	7
3.5	Failure modes on a first run	8
4	The Knowledge Pipeline	9
4.1	Scraping	9
4.1.1	The section table	9
4.1.2	Fetching	10
4.1.3	Finding chapter links	10
4.1.4	What lands on disk	10
4.2	Cleaning and chunking	11
4.2.1	Why 800 and 200	11
4.2.2	The chunking loop	11
4.2.3	Cleaning	12
4.3	Metadata: the four keys everything depends on	12
4.3.1	How categories are assigned	13
4.4	Output of the processing stage	13
5	The Retrieval Layer	15
5.1	Embeddings	15
5.2	The vector store	16
5.2.1	Indexing	16
5.2.2	Search and the where clause	16
5.2.3	Statistics	17
5.3	The retriever	17
5.3.1	Two ways to build context	18
6	The Language Model Layer	19
6.1	The informal interface	19
6.2	The four clients side by side	20
6.3	Provider selection	20
6.4	Conversation history handling	21

6.4.1	Google	21
6.4.2	OpenRouter and Ollama drop the last turn	21
6.4.3	Ollama flattens history into the prompt	22
6.5	Error handling	22
6.6	Timeouts	22
7	The Agent Layer	24
7.1	BaseAgent: the template	24
7.1.1	process(): the three-step pipeline	24
7.1.2	Retrieval with the agent’s own filter	24
7.1.3	Passing structured findings between agents	25
7.2	The three specialists	25
7.2.1	What each prompt demands	26
7.3	The orchestrator	26
7.3.1	Routing	26
7.3.2	The two paths	27
7.3.3	The cascade	27
7.3.4	Source extraction	28
7.3.5	Synthesis	29
7.4	A full slow-path trace	30
7.5	Cost of one consultation	31
8	The Interface	32
8.1	Wiring	32
8.2	The chat handler	33
8.3	Layout	34
8.4	Event wiring	34
8.5	Launching	35
9	Packaging and Deployment	36
9.1	Hugging Face Spaces	36
9.1.1	Getting the vector database into the Space	36
9.2	Docker	37
9.2.1	Compose	37
9.3	Local sharing	38
9.4	Resource notes	38
10	The Evaluation Harness	39
10.1	The dataset	39
10.2	The three systems compared	40
10.2.1	How Prakriti is extracted from free text	40
10.2.2	An inconsistency between the two drivers	41
10.3	Metrics	41
10.3.1	What the automated completeness score really measures	41
10.4	Results	42
10.4.1	Reading these numbers honestly	42
10.5	Running it	43
11	Where the Documentation and the Code Disagree	44
11.1	Claims that are no longer true	45
11.2	Defects found while reading	45
11.3	Smaller inconsistencies	47
11.4	If you fix three things, fix these	47

12 Extending the System	48
12.1 Adding a fourth specialist	48
12.1.1 1. Tag the corpus	48
12.1.2 2. Write the agent	48
12.1.3 3. Teach the orchestrator about it	49
12.1.4 4. Wire it up	49
12.2 Adding a new LLM provider	49
12.3 Adding a second source text	50
12.4 Making the source filter work	51
12.5 Things that are harder than they look	51
A Reference	52
A.1 Public API by module	52
A.2 The result dictionary	53
A.3 Hand-written test questions	53
A.4 Commit history worth knowing	54
A.5 Glossary	54

What AyurMind Is

AyurMind answers health questions in the vocabulary of classical Ayurveda, and it tries to answer them from a book rather than from memory. The book is the Charaka Samhita, scraped from carakasamhitaonline.com, cut into roughly 800-token passages, embedded with a sentence-transformer model, and stored in a local ChromaDB collection called `ayurvedic_texts`. When you ask a question, the system searches that collection, hands the matching passages to a language model along with a role-specific instruction, and assembles the answers into a consultation report.

The part that makes it more than a plain retrieval chatbot is the split into four roles. Three specialist agents each own one clinical task and each search a different slice of the corpus. A fourth agent, the orchestrator, decides which specialists to wake up, feeds each one's output into the next, and writes the final report. A question about diet alone touches one agent; a question describing symptoms, constitution, and a request for a remedy touches all three.

1.1 The problem it targets

Two things go wrong when a general-purpose language model is asked an Ayurvedic question. It invents plausible-sounding Sanskrit that has no textual basis, and it flattens the distinction between constitution (*Prakriti*, what you were born with) and current imbalance (*Vikriti*, what is wrong right now). A treatment that ignores that distinction can be actively wrong: a warming, oily regimen that settles Vata will aggravate a Pitta patient.

The design responds to both. Grounding is handled by retrieval — every specialist prompt carries retrieved passages and the model is told to answer from them. The Prakriti/Vikriti distinction is handled structurally, by giving each concept its own agent, its own system prompt, and its own metadata filter over the corpus.

1.2 Who the output is written for

The system prompts are unusually specific about audience. All three specialists and the orchestrator are told they are addressing BAMS graduates and practitioners, and are instructed to use Sanskrit clinical terminology, cite the classical section a recommendation comes from, and drop conversational framing. From `src/agents/orchestrator.py`:

```
-Precision over Personability: Prioritize clinical accuracy and detailed
  explanations over conversational pleasantries. Avoid "friendly" or
  "approachable" language. Do not use phrasings like "Let me explain..."
```

That choice explains a lot of the code. It is why the synthesis prompt enforces a four-heading report structure, why the treatment agent must attach a supervision warning to any Panchakarma procedure, and why the evaluation harness asks BAMS practitioners rather than crowdworkers to score the outputs.

1.3 The two paths through the system

Every query hits `OrchestratorAgent.simple_query()`, which does a keyword scan and then picks one of two routes.

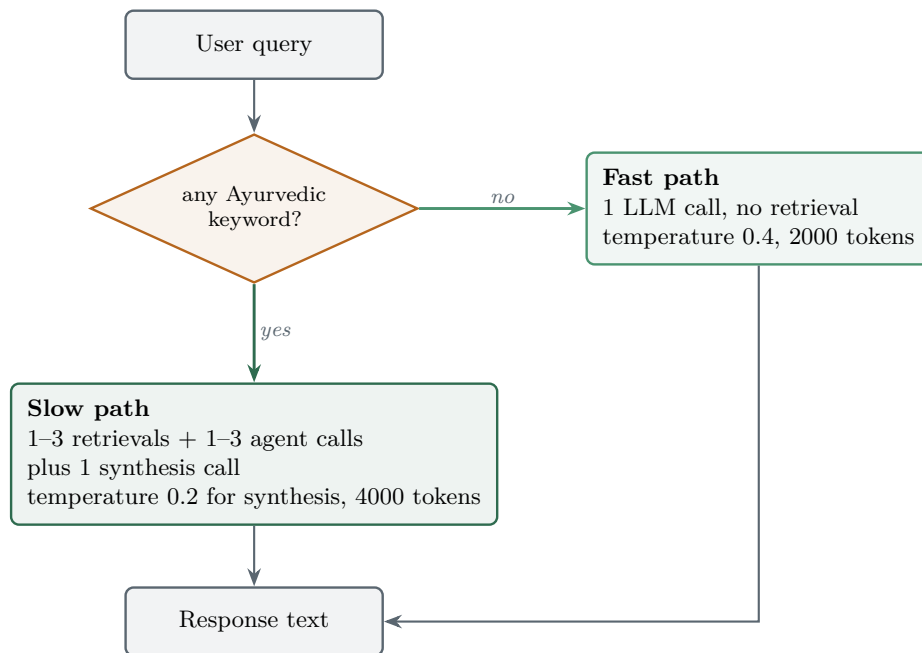


Figure 1.1: Query routing. The keyword scan in `analyze_query()` is the only gate; there is no model-based classifier.

The fast path exists for cost and latency. Asking “what is Ayurveda?” does not need three retrievals and four model calls, so it gets one call with a short, friendly system prompt. Everything else — anything mentioning a dosha, a symptom, or a remedy — takes the slow path.

1.4 Technology in one table

Concern	Choice	Where it lives
Source text	Charaka Samhita, 8 sections	<code>src/scrapper/charaka_scraper.py</code>
Chunking	800 tokens, 200 overlap, <code>cl100k_base</code>	<code>src/scrapper/data_processor.py</code>
Embeddings	<code>all-mpnet-base-v2</code> (768-dim)	<code>src/rag/embeddings.py</code>
Vector store	ChromaDB, persistent on disk	<code>src/rag/vectorstore.py</code>
Default LLM	Gemini <code>gemini-1.5-flash</code>	<code>src/llm/google_client.py</code>
Alternate LLMs	OpenAI, OpenRouter, Ollama	<code>src/llm/</code>
Agents	3 specialists + 1 orchestrator	<code>src/agents/</code>
Interface	Gradio Blocks, port 7860	<code>src/ui/gradio_app.py</code>
Packaging	Dockerfile + compose, HF Spaces	<code>Dockerfile</code> , <code>app.py</code>

Table 1.1: The stack as the code actually configures it.

The README describes an older design

`README.md` names OpenRouter as the primary LLM provider, lists LangChain as the RAG framework, and shows Sushruta Samhita and Ashtanga Hridaya in the knowledge base. None of those are true of the current code: the default provider is Google Gemini, `langchain` is never imported despite being pinned in `requirements.txt`, and the scraper only covers Charaka. Chapter 11 lists every divergence with the file that proves it. Read that chapter before trusting the README.

Repository Layout

Seventy files are tracked in git. Roughly a third of them are source, a third are evaluation artefacts, and a third are documentation and diagrams. The tree below marks what each directory is responsible for.

```
AyurMind/
|
|-- app.py entry point for Hugging Face Spaces
|-- Dockerfile python:3.10-slim image, CMD -> scripts/04_run_app.py
|-- docker-compose.yml single service, port 7860, ./data mounted
|-- requirements.txt pinned deps (some unused -see chapter 11)
|-- requirements_simplified.txt smaller variant, no langchain/jupyter
|
|-- scripts/ the four-step pipeline, run in order
| |-- 01_scrape_data.py scrape + chunk (15-30 min)
| |-- 02_build_vectordb.py embed + index (5-10 min)
| |-- 03_test_rag.py three smoke queries (1 min)
| |-- 04_run_app.py launch Gradio (instant)
|
|-- src/
| |-- scraper/
| | |-- charaka_scraper.py CharakaScraper: fetch 8 sections -> data/raw/
| | |-- data_processor.py AyurvedicTextProcessor: clean, chunk, tag
| |-- rag/
| | |-- embeddings.py EmbeddingGenerator: sentence-transformers
| | |-- vectorstore.py AyurvedicVectorStore: ChromaDB wrapper
| | |-- retriever.py RAGRetriever: embed query -> search -> chunks
| |-- llm/
| | |-- google_client.py GoogleClient (default)
| | |-- openai_client.py OpenAIClient (USE_OPENAI=true)
| | |-- local_client.py OllamaClient (USE_LOCAL_LLM=true)
| | |-- openrouter_client.py OpenRouterClient (present, never imported)
| |-- agents/
| | |-- base_agent.py BaseAgent: retrieve -> prompt -> generate
| | |-- prakriti_agent.py constitution, category filter "prakriti"
| | |-- dosha_agent.py imbalance, category filter "vikriti"
| | |-- treatment_agent.py therapy, category filter "treatment"
| | |-- orchestrator.py routing, cascade, synthesis
| |-- ui/
| |-- gradio_app.py AyurMindApp: wiring + chat handler
|
|-- evaluation/
| |-- dataset/evaluation_cases.csv 40 labelled scenarios
| |-- results/*.jsonl raw model outputs, 40 lines each
| |-- run_evaluation.py driver (currently vanilla only)
| |-- run_evaluation_REAL.py driver for the full multi-agent run
| |-- calculate_metrics.py automated metrics + review templates
| |-- generate_expert_templates.py BAMS-facing CSV templates
| |-- evaluation_summary_report.md generated results table
|
|-- images/ rendered PNGs of the two mermaid diagrams
|-- *.mermaid, *.mmd architecture and dataflow diagram sources
|-- diagrams.html standalone mermaid viewer
|-- prompts.txt 20 hand-written test questions by category
|-- research_proposal.js 70 KB docx generator for the conference paper
```

2.1 Directories the code creates but git ignores

`.gitignore` excludes `data/` entirely, so nothing under it is in the repository. Three directories appear at runtime:

Path	Created by	Contents
data/raw/	CharakaScraper	One folder per section: <code>index.html</code> , <code>chapter_NN.html</code> , matching <code>.txt</code> files, <code>metadata.json</code> . Plus <code>scraping_summary.json</code> at the top.
data/processed/	AyurvedicTextProcessor	<code>all_chunks.json</code> (every chunk with text and metadata) and <code>processing_summary.json</code> .
data/vectordb/	AyurvedicVectorStore	ChromaDB's SQLite file and index segments. Around 100 MB once populated.

Table 2.1: Runtime data directories. A fresh clone has none of these; step 1 and step 2 of the pipeline build them.

The vector database is the deployment artefact

Everything before `scripts/02_build_vectordb.py` is a one-off. Once `data/vectordb/` exists you can throw away `data/raw/` and `data/processed/` — the application only ever reads the Chroma collection. This matters for Docker and Hugging Face Spaces, where you want to ship the built database rather than scrape on every cold start.

2.2 Files the README promises that do not exist

`README.md` prints a project tree that includes a `tests/` package with three test modules, a `config/` directory holding `config.yaml` and `prompts.yaml`, a `notebooks/` folder, plus `.env.example`, `LICENSE`, and `CONTRIBUTING.md`. None of these are tracked in git. Prompts live inline as Python strings inside each agent class; configuration is read from environment variables through `os.getenv` at the point of use; there is no test suite, so the `pytest tests/` instruction in the README fails immediately.

2.3 Reading order for a newcomer

If you are opening this repository for the first time, the shortest path to understanding it is bottom-up, following the direction data actually moves:

1. `src/scrapper/data_processor.py` — see what a chunk looks like and what metadata it carries. Everything downstream depends on those four metadata keys.
2. `src/rag/vectorstore.py` — 53 lines, and it shows exactly how the metadata is turned into a Chroma `where` filter.
3. `src/agents/base_agent.py` — 63 lines that define the entire agent contract.
4. `src/agents/orchestrator.py` — the only file with real control flow.
5. `src/ui/gradio_app.py` — the wiring that connects the four above.

The four LLM clients can be skimmed; they are near-identical adapters around different HTTP APIs.

CHAPTER 3

Getting It Running

3.1 What you need

Python 3.9 or newer, around 8 GB of RAM, and roughly 5 GB of free disk. The RAM figure is driven by `all-mpnet-base-v2`, which loads into memory at startup and stays there; the disk figure is the scraped HTML plus the Chroma index. You also need one of: a Google AI API key, an OpenAI API key, or a local Ollama installation.

3.2 Install

```
git clone https://github.com/charansaiponnada/AyurMind.git
cd AyurMind

python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # macOS / Linux

pip install -r requirements.txt
```

`requirements.txt` pins 25 packages including Jupyter, black, flake8, mypy and pytest. For a runtime-only install, `requirements_simplified.txt` drops the development tooling and the unused LangChain packages.

3.3 Configuration

There is no config file and no `.env.example` in the repository. Create `.env` yourself in the project root; `python-dotenv` loads it from both `src/ui/gradio_app.py` and `src/scrapper/charaka_scraper.py`. A minimal working file is three lines:

```
GOOGLE_API_KEY=your_key_here
GOOGLE_MODEL=gemini-1.5-flash
GRADIO_PORT=7860
```

Every variable the code reads, with the default it falls back to:

Variable	Default	Read by
<i>Provider selection</i>		
USE_LOCAL_LLM	false	AyurMindApp.__init__ — tries Ollama first
USE_OPENAI	false	AyurMindApp.__init__ — tries OpenAI second
<i>Google (the default provider)</i>		
GOOGLE_API_KEY	—	GoogleClient; raises ValueError if absent
GOOGLE_MODEL	gemini-1.5-flash	GoogleClient
<i>OpenAI</i>		
OPENAI_API_KEY	—	OpenAIClient
OPENAI_MODEL	gpt-4o-mini	OpenAIClient
<i>Ollama</i>		
LOCAL_MODEL	llama3.2:3b	OllamaClient
<i>OpenRouter (client exists, app never selects it)</i>		
OPENROUTER_API_KEY	—	OpenRouterClient
DEFAULT_MODEL	mistralai/mistral-7b-instruct:free	OpenRouterClient
<i>Retrieval</i>		
EMBEDDING_MODEL	sentence-transformers/all-mpnet-base-v2	EmbeddingGenerator
VECTOR_DB_PATH	./data/vectordb	AyurvedicVectorStore
MAX_CHUNKS_PER_QUERY	5	RAGRetriever
<i>Agents</i>		
ORCHESTRATOR_TEMP	0.2	OrchestratorAgent, synthesis only
<i>Scraper</i>		
REQUEST_DELAY	2	seconds between HTTP requests
MAX_RETRIES	3	attempts per page
TIMEOUT	30	seconds per request
<i>Interface</i>		
GRADIO_PORT	7860	AyurMindApp.launch
GRADIO_SHARE	false	AyurMindApp.launch; also flips bind address

Table 3.1: Complete environment variable reference, gathered from every `os.getenv` call in the codebase.

Agent temperatures are hard-coded

Only the orchestrator’s synthesis temperature is configurable. The three specialists set theirs in their constructors — 0.3 for Prakriti and Dosha, 0.4 for Treatment — and the fast path uses a literal 0.4. Changing those means editing the agent files.

3.4 The four-step pipeline

The scripts are numbered because each one consumes what the previous one produced. Running them out of order fails loudly rather than silently.

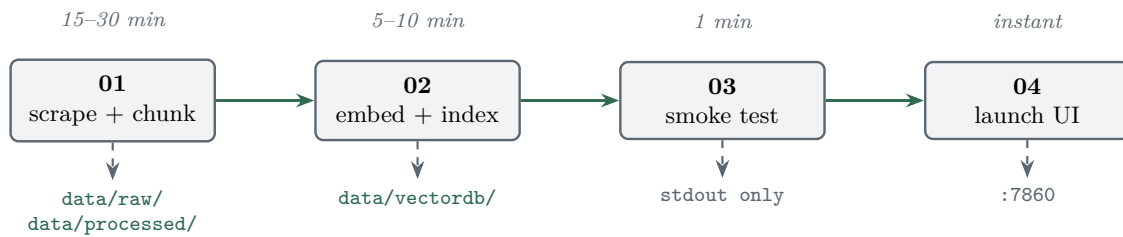


Figure 3.1: Pipeline stages with their runtimes and outputs. Only steps 1 and 2 write to disk.

```
python scripts/01_scrape_data.py # -> data/raw/ and data/processed/
python scripts/02_build_vectordb.py # -> data/vectordb/
python scripts/03_test_rag.py # prints three retrieval results
python scripts/04_run_app.py # http://127.0.0.1:7860
```

Step 4 accepts a flag:

```
python scripts/04_run_app.py --share
```

which sets Gradio's `share=True`, binds to `0.0.0.0` and produces a temporary public `gradio.live` URL. Setting `GRADIO_SHARE=true` in `.env` does the same thing.

3.4.1 What step 1 actually prints

`scripts/01_scrape_data.py` runs two phases in one process — `CharakaScraper.scrape_all()` then `AyurvedicTextProcessor.process_all()` — and reports counts from both:

```

36     print("SCRAPING COMPLETE!")
37     print(f"Scraped {scraping_summary['total_sections']} sections")
38     print(f"Total chapters: {scraping_summary['total_chapters']}")
39     print(f"Total words: {scraping_summary['total_words']:,}")
40     ...
41     print(f"Created {processing_summary['total_chunks']:,} chunks")
42     print(f"Total tokens: {processing_summary['total_tokens']:,}")
43     print(f"Avg chunk size: {processing_summary['avg_chunk_size']:.0f} tokens")
44     print("Category breakdown:")
45     for category, count in processing_summary['categories'].items():
46         print(f"    - {category}: {count} chunks")

```

The category breakdown at the end is the number worth watching. It tells you how many chunks each of the three specialist agents will be able to see, and a badly skewed distribution here shows up later as an agent that retrieves nothing useful. Section 4.3.1 explains why the skew happens.

3.5 Failure modes on a first run

Symptom	Cause and fix
ModuleNotFoundError: <code>src</code>	The scripts insert <code>../src</code> into <code>sys.path</code> , but <code>evaluation/</code> scripts insert the project <i>root</i> instead. Run scripts from the repository root.
Run <code>01_scrape_data.py</code> first!	<code>scripts/02_build_vectordb.py</code> checks for <code>data/processed/all_chunks.json</code> and exits with code 1 when it is missing.
ValueError: Google API key not found	No <code>.env</code> , or the file is not in the directory you launched from. <code>load_dotenv()</code> searches upward from the current working directory.
Retrieval returns zero chunks	The Chroma collection is empty. <code>AyurvedicVectorStore.get_stats()['total_chunks']</code> tells you the count in one line.
Scraper stalls or returns empty pages	<code>REQUEST_DELAY</code> is 2 seconds by default and the scraper doubles it on retry. The chapter-link extractor is tied to the site's MediaWiki markup (<code>div#mw-content-text</code>); a site redesign breaks it silently, yielding zero chapters rather than an error.

Table 3.2: The errors you are most likely to see, and where they come from.

The Knowledge Pipeline

Two classes turn a MediaWiki site into a searchable corpus. `CharakaScraper` downloads and strips HTML; `AyurvedicTextProcessor` cleans, splits, and tags the result. They run back to back inside `scripts/01_scrape_data.py`.

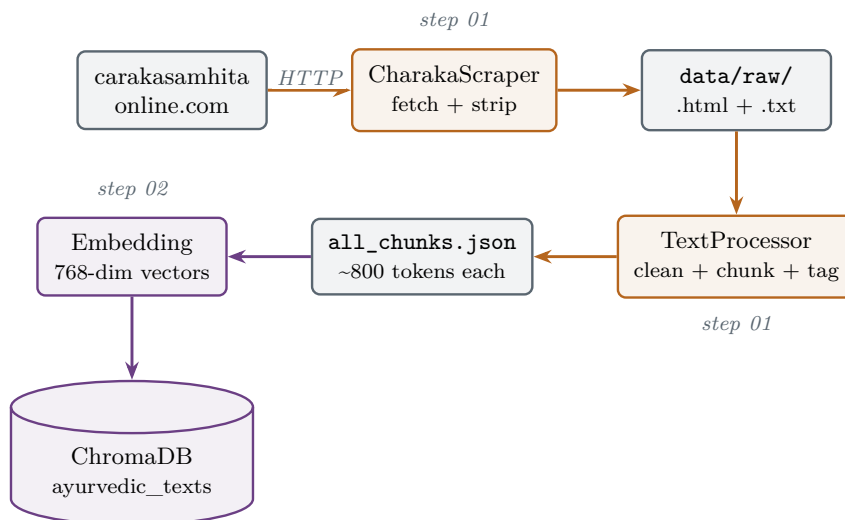


Figure 4.1: Ingestion, end to end. The first three boxes run once; the Chroma collection is what the application actually queries.

4.1 Scraping

4.1.1 The section table

The eight books of the Charaka Samhita are hard-coded as a class attribute. Each entry carries a display name, the MediaWiki URL, and a Roman numeral used later as `section_code` metadata.

```

34  SECTIONS = {
35      "Sutra_Sthana": {
36          "name": "Section on Fundamental Principles",
37          "url": "https://www.carakasamhitaonline.com/index.php?title=Sutra_Sthana",
38          "code": "I"
39      },
40      "Nidana_Sthana": {
41          "name": "Section on Diagnostic Principles",
42          "url": "https://www.carakasamhitaonline.com/index.php?title=Nidana_Sthana",
43          "code": "II"
44      },
45      # ... Vimana III, Sharira IV, Indriya V,
46      #     Chikitsa VI, Kalpa VII, Siddhi VIII
47  }
  
```

The mapping matters downstream. When a user writes “according to the Siddhi Sthana”, the orchestrator extracts that phrase and filters retrieval on the `section` metadata field, which is populated from `name` in this table.

Code	Sthana	Subject
I	Sutra	Fundamental principles: doshas, tastes, daily routine
II	Nidana	Diagnostics: causes and premonitory signs of disease
III	Vimana	Specific medical principles, including constitutional examination
IV	Sharira	Embryology, anatomy, formation of Prakriti at conception
V	Indriya	Prognosis read from sensory signs
VI	Chikitsa	Therapeutics: disease-specific management
VII	Kalpa	Pharmaceutics: preparation of Panchakarma substances
VIII	Siddhi	Procedure management and post-therapy care

Table 4.1: The eight sections and what each contributes to the corpus.

4.1.2 Fetching

`fetch_page` is deliberately slow. It sleeps `REQUEST_DELAY` seconds after every successful request and doubles that on retry, then gives up after `MAX_RETRIES`. The User-Agent identifies the crawler.

```

95     def fetch_page(self, url: str) -> Tuple[str, bool]:
96         for attempt in range(self.max_retries):
97             try:
98                 response = self.session.get(url, timeout=self.timeout)
99                 response.raise_for_status()
100                time.sleep(self.delay)           # Be respectful to the server
101                return response.text, True
102            except requests.RequestException as e:
103                logger.warning(f"Attempt {attempt + 1} failed for {url}: {e}")
104                if attempt < self.max_retries - 1:
105                    time.sleep(self.delay * 2)   # Longer delay on retry
106            logger.error(f"Failed to fetch {url} after {self.max_retries} attempts")
107            return "", False

```

The function returns a (text, ok) tuple rather than raising, so a single dead chapter logs a warning and the run continues. That is why a completed scrape can still be missing chapters — check `total_chapters` in `scraping_summary.json` against what the site lists.

4.1.3 Finding chapter links

There is no sitemap, so chapters are discovered by scanning every anchor inside the MediaWiki content div and keeping the ones that look like article links:

```

133     content_div = soup.find('div', {'id': 'mw-content-text'})
134     if content_div:
135         links = content_div.find_all('a', href=True)
136         for link in links:
137             href = link.get('href', '')
138             title = link.get_text(strip=True)
139             # Filter for actual chapter links
140             if href.startswith('/index.php?title=') and len(title) > 10:
141                 if any(skip in title.lower()
142                        for skip in ['main page', 'category', 'help', 'search']):
143                     continue
144                 chapters.append({'title': title, 'url': f"{base_url}{href}"})

```

Two heuristics carry the whole thing: the href prefix, and a link text longer than ten characters. Both are guesses about how the site is built. If `carakasamhitaonline.com` changes its markup, `extract_chapter_links` returns an empty list, the section directory gets an `index.html` and nothing else, and the run reports success. That silent failure is the most likely way a fresh scrape produces a thin corpus.

4.1.4 What lands on disk

```

data/raw/
|-- scraping_summary.json totals across all 8 sections
|-- Sutra_Sthana/
| |-- index.html index.txt the section landing page
| |-- chapter_01.html chapter_01.txt
| |-- chapter_02.html chapter_02.txt
| |-- ...
| '-- metadata.json per-chapter titles, URLs, word counts
|-- Nidana_Sthana/
'-- ...

```

Both HTML and plain text are kept. The text is what feeds the processor; the HTML is insurance against needing to re-extract without re-crawling.

4.2 Cleaning and chunking

4.2.1 Why 800 and 200

AyurvedicTextProcessor defaults to 800-token chunks with 200 tokens of overlap, counted with tiktoken's cl100k_base encoding. Eight hundred tokens is roughly two or three verses plus their commentary — enough that a retrieved passage carries its own explanation. The 200-token overlap means a statement that straddles a boundary still appears whole in one of the two neighbouring chunks.

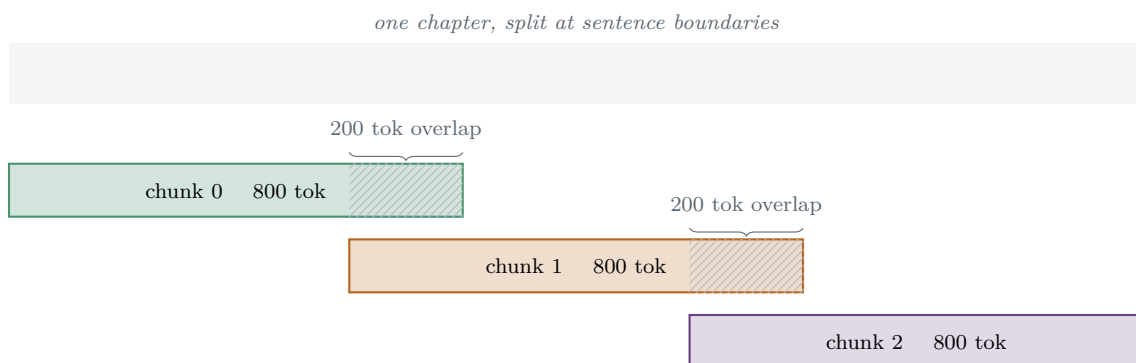


Figure 4.2: The overlap window. Hatched regions are duplicated between neighbouring chunks, so a sentence never falls into the gap between two retrievable passages.

4.2.2 The chunking loop

Text is split into sentences with a lookbehind regex, then sentences are accumulated until the token budget is hit. The interesting part is what happens at the boundary — the loop walks backwards through the chunk it just closed, collecting whole sentences until it has 200 tokens of context, and seeds the next chunk with them.

```

134     # Split into sentences
135     sentences = re.split(r'(?<=[.!?])\s+', text)
136
137     current_chunk = []
138     current_tokens = 0
139     chunk_id = 0
140
141     for sentence in sentences:
142         sentence = sentence.strip()
143         if not sentence:
144             continue
145
146         sentence_tokens = self.count_tokens(sentence)
147
148         # Check if adding this sentence exceeds chunk size

```

```

149         if current_tokens + sentence_tokens > self.chunk_size and current_chunk:
150             chunks.append({
151                 'chunk_id':      chunk_id,
152                 'text':          ' '.join(current_chunk),
153                 'token_count':   current_tokens,
154                 'metadata':      metadata.copy()
155             })
156             chunk_id += 1
157
158             # Start new chunk with overlap: keep trailing sentences
159             overlap_sentences, overlap_tokens = [], 0
160             for sent in reversed(current_chunk):
161                 sent_tokens = self.count_tokens(sent)
162                 if overlap_tokens + sent_tokens <= self.chunk_overlap:
163                     overlap_sentences.insert(0, sent)
164                     overlap_tokens += sent_tokens
165                 else:
166                     break
167
168             current_chunk = overlap_sentences
169             current_tokens = overlap_tokens
170
171             current_chunk.append(sentence)
172             current_tokens += sentence_tokens

```

Because overlap is measured in whole sentences, the actual overlap is usually a little under 200 tokens — the loop stops as soon as the next sentence backwards would exceed the budget. A chunk is never split mid-sentence, which is the point.

The sentence splitter and Sanskrit transliteration

`re.split(r'(?<=[.!?])\s+', text)` treats every period as a sentence end. Transliterated Sanskrit and the site's verse numbering ("Ch. Su. 1.42") contain periods that are not sentence boundaries, so some chunks begin mid-citation. The overlap window softens this but does not eliminate it.

4.2.3 Cleaning

`clean_text` runs before chunking and does four things: collapses runs of three or more newlines to two, collapses multiple spaces to one, deletes [Page N] markers and bare numeric lines, and normalises curly quotes to straight ones.

```

69     text = re.sub(r'\n\s*\n\s*\n', '\n\n', text)
70     text = re.sub(r' +', ' ', text)
71     text = re.sub(r'\[Page \d+\]', '', text)
72     text = re.sub(r'^\d+\s*$', '', text, flags=re.MULTILINE)

```

4.3 Metadata: the four keys everything depends on

Each chunk carries a metadata dictionary. Four keys are load-bearing:

Key	Example	Used for
section	"Section on Therapeutic Principles"	Source filtering when a user names a Sthana; printed in every citation
section_code	"VI"	Not queried, kept for reference
chapter	"Jvara Chikitsa"	Second half of every citation line
category	"treatment"	The agent routing key. Each specialist filters on exactly one value

Table 4.2: Chunk metadata. `category` is what makes the three agents see three different corpora.

4.3.1 How categories are assigned

`categorize_content` is a keyword cascade over the lowercased chunk text, with a fall-through to section-based assignment:

```

226     text_lower = text.lower()
227
228     # Prakriti-related content
229     if any(term in text_lower for term in
230            ['vata', 'pitta', 'kapha', 'constitution', 'prakriti']):
231         return 'prakriti'
232
233     # Vikriti/Disease content
234     if any(term in text_lower for term in
235            ['disease', 'symptom', 'disorder', 'imbalance', 'dosha']):
236         return 'vikriti'
237
238     # Treatment content
239     if any(term in text_lower for term in
240            ['treatment', 'therapy', 'remedy', 'herb', 'diet', 'lifestyle']):
241         return 'treatment'
242
243     # Default to section-based categorization
244     if 'chikitsa' in section_name.lower():
245         return 'treatment'
246     elif 'nidana' in section_name.lower() or 'vimana' in section_name.lower():
247         return 'vikriti'
248     else:
249         return 'general'

```

Order decides everything, and the order is not neutral. The three dosha names are in the first test, and the three dosha names appear on a large share of pages in any Ayurvedic text. A passage from the Chikitsa Sthana that reads “for aggravated Vata, administer Basti” contains `vata` and so is tagged `prakriti` — not `treatment` — even though it is a treatment instruction sitting in the therapeutics book. The treatment agent, which filters on `category == "treatment"`, will never retrieve it.

Consequence: the treatment corpus is smaller than it looks

Because `prakriti` is tested first and its keywords are the most common words in the corpus, the `prakriti` bucket absorbs passages that belong to the other two agents. Run the category breakdown printed at the end of step 1 and check the split before drawing conclusions about agent quality. A more balanced assignment would score all three keyword sets and pick the highest, or tag chunks with multiple categories and use Chroma’s `$in` operator at query time.

4.4 Output of the processing stage

`process_all()` writes two files. `all_chunks.json` is a flat list of every chunk:

```

[
  {
    "chunk_id": 0,
    "text": "Vata is characterised by the qualities ruksha, laghu, sheeta ...",
    "token_count": 782,
    "metadata": {
      "section": "Section on Fundamental Principles",
      "section_code": "I",
      "chapter": "Deerghanjiviteeya Adhyaya",
      "chapter_number": 1,
      "source_url": "https://www.carakasamhitaonline.com/index.php?title=...",
      "source_file": "chapter_01.txt",
      "source_path": "data/raw/Sutra_Sthana/chapter_01.txt",
      "category": "prakriti"
    }
  }
]

```

`processing_summary.json` holds the totals, the per-section chunk and token counts, and the category histogram. The README estimates 5,000–8,000 chunks for the full corpus.

The Retrieval Layer

Three small classes sit between a question and the passages that answer it. `EmbeddingGenerator` turns text into vectors, `AyurvedicVectorStore` owns the Chroma collection, and `RAGRetriever` joins them. Together they are under 130 lines. No framework is involved — despite `langchain` being pinned in `requirements.txt`, it is never imported.

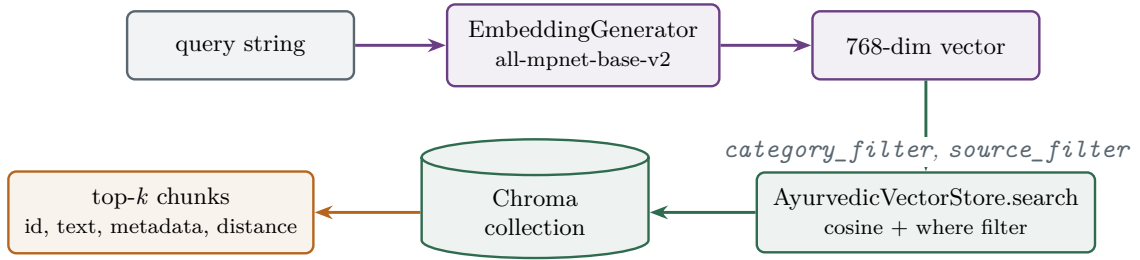


Figure 5.1: One retrieval. The two filters are optional and are combined into a single Chroma `where` clause.

5.1 Embeddings

```

1 class EmbeddingGenerator:
2     def __init__(self, model_name: str = None):
3         if model_name is None:
4             model_name = os.getenv("EMBEDDING_MODEL",
5                                   "sentence-transformers/all-mpnet-base-v2")
6         self.model = SentenceTransformer(model_name)
7         self.embedding_dim = self.model.get_sentence_embedding_dimension()
8
9     def embed_text(self, text: str) -> np.ndarray:
10        return self.model.encode(text, convert_to_numpy=True)
11
12    def embed_batch(self, texts: List[str], batch_size: int = 32) -> np.ndarray:
13        return self.model.encode(texts, batch_size=batch_size,
14                                  show_progress_bar=True, convert_to_numpy=True)

```

`all-mpnet-base-v2` produces 768-dimensional vectors and is a general-purpose English model. It has never seen Sanskrit, so transliterated terms are embedded as unfamiliar token sequences. Retrieval works because the scraped text is an English translation with Sanskrit terms interleaved — the English carries the semantic signal. Queries written mostly in transliterated Sanskrit will retrieve worse than the same question asked in English.

The class is instantiated twice in a normal session: once by `scripts/02_build_vectordb.py` to embed the corpus, once by `AyurMindApp` to embed queries. Both load the same model weights, roughly 420 MB, into memory.

Why `embed_batch` exists separately

Indexing embeds thousands of chunks at once, which is far faster in batches of 32 with a progress bar. Query time embeds exactly one string, where the batching machinery is pure overhead. Hence two methods over the same model.

5.2 The vector store

AyurvedicVectorStore wraps a single persistent Chroma collection.

```

9     def __init__(self, persist_directory: str = None):
10         if persist_directory is None:
11             persist_directory = os.getenv("VECTOR_DB_PATH", "./data/vectordb")
12         self.persist_directory = Path(persist_directory)
13         self.persist_directory.mkdir(parents=True, exist_ok=True)
14
15         self.client = chromadb.PersistentClient(path=str(self.persist_directory))
16         self.collection = self.client.get_or_create_collection(
17             name="ayurvedic_texts",
18             metadata={"description": "Charaka Samhita chunks"}
19         )

```

`get_or_create_collection` makes the constructor safe to call against an empty directory — you get an empty collection rather than an exception. That is convenient and also the reason a misconfigured `VECTOR_DB_PATH` produces zero search results instead of a clear error.

5.2.1 Indexing

```

21     def add_chunks(self, chunks: List[Dict], embeddings: List[List[float]],
22                     batch_size: int = 100):
23         for i in tqdm(range(0, len(chunks), batch_size), desc="Adding chunks"):
24             batch_chunks = chunks[i:i+batch_size]
25             batch_embeddings = embeddings[i:i+batch_size]
26
27             ids = [f"chunk_{i+j}" for j in range(len(batch_chunks))]
28             documents = [chunk['text'] for chunk in batch_chunks]
29             metadatas = [chunk['metadata'] for chunk in batch_chunks]
30
31             self.collection.add(ids=ids, documents=documents,
32                                embeddings=batch_embeddings, metadatas=metadatas)

```

IDs are positional — `chunk_0` through `chunk_N`, derived from the offset in `all_chunks.json`. They carry no meaning and are not stable across rebuilds: re-running the scraper with a different chapter count rennumbers everything. Nothing in the application relies on ID stability, but the evaluation results files record source IDs, so those IDs only make sense against the database version they were produced with.

Re-indexing does not clear the collection

`add_chunks` only calls `collection.add`. Running `scripts/02_build_vectordb.py` twice against an existing database attempts to insert the same IDs again. Delete `data/vectordb/` before a rebuild.

5.2.2 Search and the where clause

This is the most consequential method in the retrieval layer, because it is where the agents' category filters and the orchestrator's source filter get combined.

```

32     def search(self, query_embedding: List[float], n_results: int = 5,
33                category_filter: Optional[str] = None,
34                source_filter: Optional[str] = None) -> Dict:
35         where_conditions = []
36         if category_filter:
37             where_conditions.append({"category": category_filter})
38
39         if source_filter:
40             # Match source_filter against 'section' or 'chapter' metadata
41             where_conditions.append({"$or": [{"section": source_filter},
42                                             {"chapter": source_filter}]})
43
44         where_clause = None

```

```

45     if where_conditions:
46         if len(where_conditions) == 1:
47             where_clause = where_conditions[0]
48         else:
49             where_clause = {"$and": where_conditions}
50
51     return self.collection.query(query_embeddings=query_embedding,
52                                n_results=n_results, where=where_clause)

```

Three shapes come out of that logic:

```

// no filters -the fast path and unfiltered helper calls
where = None

// category only -the normal case for every specialist agent
where = {"category": "treatment"}

// category + source -user named a Sthana in the query
where = {"$and": [
    {"category": "treatment"},
    {"$or": [{"section": "Siddhi Sthana"},
             {"chapter": "Siddhi Sthana"}]}
]}

```

The source filter needs an exact string match

Chroma's `where` does equality, not substring matching. The section values written by the processor are full display names — "Section on Therapeutic Procedures" — while the orchestrator's regex extracts "Siddhi Sthana". Those strings are not equal, so the `$or` matches nothing, the `$and` matches nothing, and a query that names a Sthana retrieves **zero** chunks rather than more focused ones. Fixing this means either storing the Sthana key ("Siddhi_Sthana" → "Siddhi Sthana") as its own metadata field, or mapping the extracted phrase to the full display name before filtering.

5.2.3 Statistics

```

51     def get_stats(self) -> Dict:
52         total_count = self.collection.count()
53         return {'total_chunks': total_count, 'categories': {}}

```

`categories` is a placeholder that always returns empty. The real category histogram is in `data/processed/processing_summary.json`, written during step 1.

5.3 The retriever

`RAGRetriever` owns the default result count and reshapes Chroma's column-oriented response into a list of dictionaries.

```

13     def retrieve(self, query: str, n_results: int = None,
14                 category_filter: Optional[str] = None,
15                 source_filter: Optional[str] = None) -> List[Dict]:
16         if n_results is None:
17             n_results = self.max_chunks # MAX_CHUNKS_PER_QUERY, default 5
18
19         query_embedding = self.embedding_generator.embed_text(query)
20         results = self.vectorstore.search(
21             query_embedding=query_embedding.tolist(), n_results=n_results,
22             category_filter=category_filter, source_filter=source_filter)
23
24         retrieved_chunks = []
25         for i in range(len(results['ids'][0])):
26             retrieved_chunks.append({
27                 'id': results['ids'][0][i],
28                 'text': results['documents'][0][i],
29                 'metadata': results['metadatas'][0][i],

```

```

30         'distance': results['distances'][0][i]
31         if 'distances' in results else None
32     })
33     return retrieved_chunks

```

The [0] indexing everywhere is Chroma's batch dimension — `query` accepts a list of query vectors and returns parallel lists of results. One query goes in, so index zero comes out.

Note what is *not* here: no distance threshold, no reranking, no deduplication, no minimum-relevance cutoff. Whatever the nearest five vectors are, they go into the prompt. If the collection holds nothing relevant, the five least-irrelevant chunks are still retrieved and still presented to the model as authoritative context. This is the main structural weakness of the grounding story, and it is worth knowing when reading the hallucination-rate discussion in Chapter 10.

5.3.1 Two ways to build context

`RAGRetriever.build_context()` formats chunks into a prompt-ready string. It is used by the single-agent RAG baseline in the evaluation harness:

```

32     def build_context(self, query: str, n_results: int = None,
33                       category_filter: Optional[str] = None,
34                       include_metadata: bool = True) -> str:
35         chunks = self.retrieve(query, n_results, category_filter)
36         context_parts = []
37         for i, chunk in enumerate(chunks, 1):
38             if include_metadata:
39                 metadata = chunk['metadata']
40                 source = (f"[Source: {metadata.get('section', 'Unknown')}] - "
41                          f"{metadata.get('chapter', 'Unknown')}]")
42                 context_parts.append(f"--- Context {i} {source} ---")
43             else:
44                 context_parts.append(f"--- Context {i} ---")
45                 context_parts.append(chunk['text'])
46                 context_parts.append("")
47         return "\n".join(context_parts)

```

The agents do not call it. `BaseAgent.retrieve_context_and_sources()` produces the same string but also returns the raw chunk list, because the orchestrator needs the chunk metadata to build its citation block. The duplication is small but real: two functions format context identically, and `build_context` has no `source_filter` parameter, so it cannot honour a user's request for a specific Sthana.

The resulting context string looks like this:

```

---Context 1 [Source: Section on Therapeutic Principles -Jvara Chikitsa] ---
In the management of Jvara, the physician should first assess the state of
Agni. Langhana is indicated in Ama-avastha ...

---Context 2 [Source: Section on Fundamental Principles -Deerghanjiviteeya] ---
...

```

The Language Model Layer

Four client classes live in `src/llm/`. They target different APIs but expose the same two methods, so any of them can be dropped into an agent without the agent knowing which one it got.

6.1 The informal interface

There is no abstract base class here — the contract is duck-typed. Every client implements:

```

1 def generate(self, prompt: str,
2               system_prompt: Optional[str] = None,
3               temperature: float = ...,
4               max_tokens: int = ...,
5               conversation_history: Optional[List[Dict]] = None,
6               **kwargs) -> str
7
8 def generate_with_context(self, query: str, context: str,
9                           system_prompt: str,
10                          temperature: float = ...,
11                          max_tokens: int = ...,
12                          conversation_history: Optional[List[Dict]] = None) -> str

```

`generate_with_context` is the RAG entry point and is the same in all four files — it wraps the retrieved passages and the question in a fixed template, then delegates to `generate`:

```

1     prompt = f"""Context from Ayurvedic texts:
2
3     {context}
4
5     ---
6
7     User Query: {query}
8
9     Based on the context provided above, please provide a response."""

```

Specialist agents call `generate_with_context`. The orchestrator calls `generate` directly, both for the fast path and for synthesis, because by then the context has already been consumed and what it is passing along is the specialists' prose.

6.2 The four clients side by side

Client	Default model	Selected by	Notes
GoogleClient	gemini-1.5-flash	fallback, always last	The de facto default. <code>google-generativeai</code> SDK, stateless <code>generate_content</code> .
OpenAIClient	gpt-4o-mini	<code>USE_OPENAI=true</code>	openai SDK. Thinnest of the four at 74 lines.
OllamaClient	llama3.2:3b	<code>USE_LOCAL_LLM=true</code>	Raw HTTP to <code>localhost:11434</code> . Free and offline; ignores <code>max_tokens</code> .
OpenRouterClient	<code>mistral-7b-instruct:free</code>	never	Raw HTTP. Not imported anywhere in the application.

Table 6.1: Client comparison. Only the first three are reachable from the running application.

6.3 Provider selection

`AyurMindApp.__init__` walks a fixed preference order, catching failures at each step:

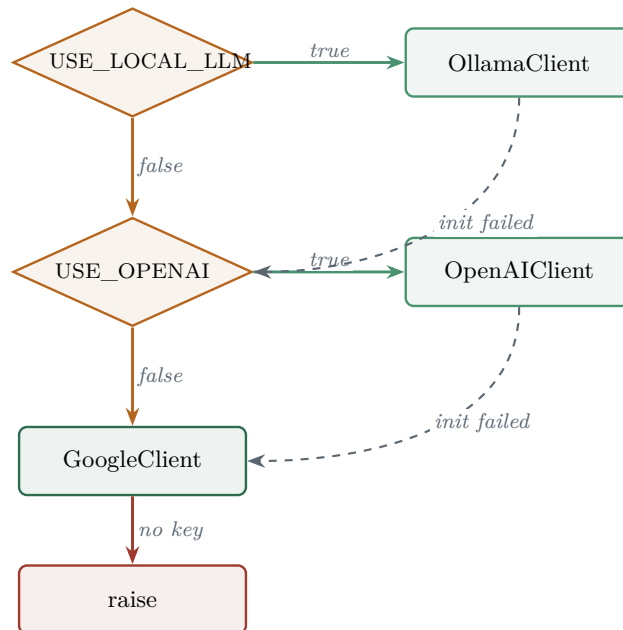


Figure 6.1: Fallback chain. A client that raises during construction is logged as a warning and the next option is tried; only a Google failure is fatal.

```

48     self.llm_client = None
49     use_local_llm = os.getenv("USE_LOCAL_LLM", "false").lower() == "true"
50     use_openai = os.getenv("USE_OPENAI", "false").lower() == "true"
51
52     if use_local_llm:
53         try:
54             self.llm_client = OllamaClient()
55             app_logger.info(f"Using Local client: {self.llm_client.model_name}")
56         except Exception as e:
57             app_logger.warning(f"Local LLM Client failed: {e}. Falling back.")
58

```

```

59     if self.llm_client is None and use_openai:
60         try:
61             self.llm_client = OpenAIClient()
62             app_logger.info(f"Using OpenAI client: {self.llm_client.model_name}")
63         except Exception as e:
64             app_logger.warning(f"OpenAI Client failed: {e}. Falling back to Google.")
65
66     if self.llm_client is None:
67         try:
68             self.llm_client = GoogleClient()
69             app_logger.info(f"Using Google client: {self.llm_client.model_name}")
70         except Exception as e:
71             app_logger.error(f"Failed to initialize GoogleClient: {e}.")
72             raise e      # Critical failure if no client can be initialized

```

The fallback covers construction only. Once a client is chosen, a mid-session API error propagates up to the Gradio handler and is shown as an error message in the chat; there is no runtime failover to a second provider.

Note also that `OllamaClient.__init__` does not raise when Ollama is unreachable — it logs a warning and returns a usable object. So `USE_LOCAL_LLM=true` with Ollama stopped selects the Ollama client successfully and then fails on the first generation call, rather than falling through to OpenAI or Google.

6.4 Conversation history handling

All four clients accept a `conversation_history` list of `{"role", "content"}` dictionaries, and this is where their behaviour genuinely diverges.

6.4.1 Google

Gemini names the assistant role `model`, so history is remapped. The client also has to cope with Gradio's message format, where `content` can be a list of part-dictionaries rather than a plain string:

```

62     def _format_history(self, conversation_history):
63         if not conversation_history:
64             return []
65         formatted_history = []
66         for turn in conversation_history:
67             # Gemini uses 'model' for the assistant's role, 'user' for user
68             role = "model" if turn["role"] == "assistant" else "user"
69             text_content = self._extract_text_content(turn["content"])
70             formatted_history.append({"role": role, "parts": [text_content]})
71         return formatted_history

```

`_extract_text_content` handles four shapes — a plain string, a list of dicts with `text` keys, a single dict with a `text` key, and anything else via `str()`. That defensiveness exists because Gradio has changed its chat message schema more than once.

6.4.2 OpenRouter and Ollama drop the last turn

Both slice the history before using it:

```

1         # OpenRouterClient.generate
2         if conversation_history:
3             # Add all but the last user message from history
4             for turn in conversation_history[:-1]:
5                 messages.append(turn)

```

That slice is correct for how the UI calls them. `AyurMindApp.chat()` appends the user's new message to `history` *before* passing it down, so the last element of `conversation_history` is the same message that arrives separately as prompt.

Google and OpenAI send the current message twice

`GoogleClient._format_history` iterates the whole list, then appends the prompt on top. `OpenAIClient` does the same. Since the UI has already appended the user's message to history, the model receives it once verbatim from history and again wrapped in the RAG or synthesis prompt. With the default Google client this happens on every slow-path turn — once per specialist agent plus once for synthesis. Adding `[:-1]` in both clients, matching the OpenRouter and Ollama behaviour, resolves it.

6.4.3 Ollama flattens history into the prompt

The Ollama `/api/generate` endpoint takes one string, so the client builds the transcript by hand:

```

66     full_prompt = ""
67     if system_prompt:
68         full_prompt += f"{system_prompt}\n\n"
69     if history_str:
70         full_prompt += f"Previous conversation:\n{history_str}\n"
71     full_prompt += f"Current query: {prompt}"
72
73     payload = {
74         "model": self.model,
75         "prompt": full_prompt,
76         "stream": False,
77         "options": {"temperature": temperature,
78                   "num_predict": 800}      # Ollama's max_tokens equivalent
79     }
```

`num_predict` is fixed at 800 regardless of what the caller asked for, and `max_tokens` is popped from `kwargs` and discarded. Agents request 3000 tokens and the orchestrator requests 4000; against Ollama they all get 800. That truncation explains a great deal about the local-model evaluation outputs in Chapter 10 — the four-section report the synthesis prompt asks for does not fit in 800 tokens.

6.5 Error handling

`OpenRouterClient` is the only one that translates HTTP status codes into actionable messages:

```

70     except requests.exceptions.HTTPError as e:
71         status_code = e.response.status_code
72         if status_code == 401:
73             raise RuntimeError("OpenRouter API Error: Invalid API Key. ...")
74         elif status_code == 402:
75             raise RuntimeError("OpenRouter API Error: You have exceeded "
76                               "your free tier limit or credits.")
77         elif status_code == 404:
78             raise RuntimeError(f"OpenRouter API Error: Model '{self.model}' "
79                               "not found. ...")
```

The other three wrap everything in a single `RuntimeError` carrying the original exception text. Google additionally treats an empty `response.text` as an error, which catches the case where Gemini's safety filters block a generation and return a response with no candidates — a realistic outcome when the prompt describes symptoms and asks for treatment.

```

142         if not response.text:
143             raise ValueError("Empty response from model")
```

6.6 Timeouts

Both HTTP clients use 180 seconds. That is generous, and deliberately so: the slow path can issue four sequential generations, and a 3B model on CPU is not fast.

Client	Timeout	Set where
OpenRouterClient	180 s	<code>requests.post(..., timeout=180)</code>
OllamaClient	180 s generate, 2 s availability probe	<code>is_available()</code> uses the short one
GoogleClient	SDK default	not overridden
OpenAIClient	SDK default	not overridden

Table 6.2: Timeout settings. A full slow-path consultation can legitimately take several minutes.

The Agent Layer

This is where the design lives. Four classes: one abstract base, three specialists that differ only in a prompt and a filter, and an orchestrator that has all the control flow.

7.1 BaseAgent: the template

BaseAgent is 63 lines and defines the entire agent contract through two abstract methods and one concrete pipeline.

```

5 class BaseAgent(ABC):
6     def __init__(self, name: str, rag_retriever, llm_client,
7         temperature: float = 0.3, max_tokens: int = 3000):
8         self.name = name
9         self.rag_retriever = rag_retriever
10        self.llm_client = llm_client
11        self.temperature = temperature
12        self.max_tokens = max_tokens
13
14        @abstractmethod
15        def get_system_prompt(self) -> str: ...
16
17        @abstractmethod
18        def get_category_filter(self) -> Optional[str]: ...

```

A subclass supplies a personality (the system prompt) and a slice of the corpus (the category filter). Everything else is inherited. Adding a fourth specialist is therefore a 15-line file — see Chapter 12.

7.1.1 process(): the three-step pipeline

```

59 def process(self, query: str, additional_info: Dict = None,
60     conversation_history: list = None) -> Dict:
61     source_filter = additional_info.get('source_filter') if additional_info else None
62     context, sources = self.retrieve_context_and_sources(query, source_filter=source_filter)
63     response = self.generate_response(query, context, additional_info, conversation_history)
64     return {'agent': self.name, 'response': response,
65         'sources': sources, 'query': query}

```

Retrieve, generate, return both the prose and the chunks it came from. Returning sources alongside response is what lets the orchestrator build a citation list without re-running retrieval.

7.1.2 Retrieval with the agent's own filter

```

21 def retrieve_context_and_sources(self, query: str, n_results: int = 5,
22     source_filter: Optional[str] = None):
23     category_filter = self.get_category_filter()
24     chunks = self.rag_retriever.retrieve(query=query, n_results=n_results,
25         category_filter=category_filter,
26         source_filter=source_filter)
27     context_parts = []
28     for i, chunk in enumerate(chunks, 1):
29         metadata = chunk.get('metadata', {})
30         source = (f"[Source: {metadata.get('section', 'Unknown')}] - "
31             f"{metadata.get('chapter', 'Unknown')}]")
32         context_parts.append(f"--- Context {i} {source} ---")
33         context_parts.append(chunk.get('text', ''))
34         context_parts.append("")
35     return "\n".join(context_parts), chunks

```

The category filter comes from the subclass and is never overridden by the caller. That is the mechanism behind the whole multi-agent premise: three agents asking the same corpus the same question get three different answers because each one is only allowed to see a third of it.

7.1.3 Passing structured findings between agents

`generate_response` folds an `additional_info` dictionary into the query as labelled lines, after removing the one key that is meant for retrieval rather than for the model:

```
43     # Ensure 'source_filter' is not included in the LLM's additional_info
44     llm_additional_info = additional_info.copy() if additional_info else {}
45     llm_additional_info.pop('source_filter', None)
46
47     if llm_additional_info:
48         info_str = "\n".join([f"{key}: {value}"
49                                for key, value in llm_additional_info.items()])
50         enhanced_query = f"{query}\n\nAdditional Information:\n{info_str}"
51     else:
52         enhanced_query = query
```

So the treatment agent's actual query text ends up looking like:

```
I have acid reflux and skin rashes. What should I do?

Additional Information:
Prakriti: **Provisional Constitutional Assessment**: Pitta-dominant, high confidence.
The user's ...
Dosha Imbalance: **Provisional Diagnosis (Vikriti Nirnaya)**: Pitta Prakopa with
Tikshnagni. Amlapitta is indicated by ...
```

Prose is passed between agents, not structured data. There is no schema, no parsing, no validation — the downstream agent reads its predecessor's report the way a junior clinician reads a colleague's notes. That keeps the code short and makes the handoff lossy in a way that is hard to measure.

One retrieval too many

`process()` calls `retrieve_context_and_sources()` and then passes the resulting context into `generate_response()`. But `generate_response()` also retrieves when context is `None`, so it carries a second, unreachable retrieval path. The dead branch is harmless; it just means two functions know how to fetch context.

7.2 The three specialists

Class	name	Temp	Filter	Persona in the system prompt
<code>PrakritiAgent</code>	Prakriti Assessor	0.3	<code>prakriti</code>	Senior professor of Sharira Vijnana
<code>DoshaAgent</code>	Dosha Imbalance Detector	0.3	<code>vikriti</code>	Senior clinician in Vikriti Vijnana
<code>TreatmentAgent</code>	Treatment Recommender	0.4	<code>treatment</code>	Senior physician designing Chikitsa Krama

Table 7.1: The three specialists. Diagnosis runs cooler than therapy design, which is given slightly more latitude at 0.4.

Each class is under 40 lines and most of that is the prompt string. The whole of `src/agents/prakriti_agent.py` outside the prompt:

```
1 class PrakritiAgent(BaseAgent):
2     def __init__(self, rag_retriever, llm_client):
3         super().__init__(name="Prakriti Assessor", rag_retriever=rag_retriever,
4                          llm_client=llm_client, temperature=0.3)
5
```

```

6     def get_system_prompt(self) -> str:
7         return """You are a senior Ayurvedic professor specializing in ..."""
8
9     def get_category_filter(self) -> Optional[str]:
10        return "prakriti"

```

7.2.1 What each prompt demands

The prompts are the specification, so they are worth reading closely. All three share four instructions — scholarly tone, classical citation, Sanskrit terminology with parenthetical glosses, and a fixed output structure — and then diverge on what they must produce.

Prakriti Assessor must return a constitutional call with a confidence level, a per-dosha breakdown naming the specific Gunas, and a trait-to-Guna correlation table. The prompt supplies the table format inline:

```

| User Trait | Assessed Dosha | Corresponding Guna |
|-----|-----|-----|
| Dry Skin | Vata | Ruksha (Dry) |
| Irritability | Pitta | Tikshna (Sharp) |

```

Dosha Imbalance Detector must go past naming a dosha and describe the pathogenesis: the nature of the vitiation (*Vata Prakopa*, *Pitta Kshaya*), which tissues and channels are involved (*Dhatus*, *Srotas*), and the state of the digestive fire (*Mandagni*, *Tikshnagni*).

Treatment Recommender has the longest prompt and the only safety-critical clause. It is instructed to lead with the treatment principle, then split recommendations into purification and palliative care, and it carries a mandatory string:

```

Advanced procedures like Vamana, Virechana, and Basti *must* include the
statement: "To be performed under the direct supervision of a qualified Vaidya."

```

It also branches on which Sthana the user asked about, which is the other half of the source-filtering feature:

```

* Kalpa Sthana: Focus on preparation and properties of specific herbs
                  for Panchakarma (e.g., Madanaphala, Vacha).
* Siddhi Sthana: Focus on management of Panchakarma procedures and
                  post-therapy care (Paschat Karma), like Samsarjana Krama.
* Chikitsa Sthana: Focus on disease-specific formulations and protocols.

```

Safety enforcement is prompt-level only

The supervision warning for Vamana, Virechana and Basti is an instruction to the model, not a post-processing check. Nothing in the code verifies that the string appears in the output before it reaches the user. The synthesis prompt asks the orchestrator to re-check it, which makes it twice as likely and still not guaranteed. A regex assertion over the final text before display would close this.

7.3 The orchestrator

7.3.1 Routing

`analyze_query` is three `any()` calls over lowercased substrings:

```

27    def analyze_query(self, query: str) -> Dict:
28        query_lower = query.lower()
29        needs_prakriti = any(term in query_lower for term in
30                             ['constitution', 'prakriti', 'vata', 'pitta', 'kapha', 'body type'])
31        needs_dosha = any(term in query_lower for term in
32                           ['symptom', 'problem', 'pain', 'disorder', 'disease', 'sick', 'imbalance'])
33        needs_treatment = any(term in query_lower for term in

```

```

34         ['treatment', 'remedy', 'cure', 'help', 'diet', 'food', 'herb', 'medicine'])
35
36         # This no longer defaults to activating all agents. If no keywords match,
37         # it correctly signals a general, non-Ayurvedic query.
38         return {'prakriti': needs_prakriti, 'dosha': needs_dosha,
39               'treatment': needs_treatment}

```

Agent	Trigger words
Prakriti	constitution, prakriti, vata, pitta, kapha, body type
Dosha	symptom, problem, pain, disorder, disease, sick, imbalance
Treatment	treatment, remedy, cure, help, diet, food, herb, medicine

Table 7.2: The complete routing vocabulary. Twenty words decide the entire execution path.

The comment in the code marks a real design change — an earlier version fired all three agents when nothing matched, and the git history shows the fix landing as *“feat: Implement smart query routing in OrchestratorAgent”*. The current behaviour costs one model call for a general question instead of four.

Two things follow from a substring match. First, ‘help’ is a treatment keyword, so “help” anywhere in a sentence activates the treatment agent — “can you help me understand Ayurveda?” takes the slow path and gets a treatment protocol it did not ask for. Second, a genuine clinical description that avoids all twenty words takes the fast path with no retrieval at all: “I wake at 3am with a racing mind and my joints crack” contains none of them.

7.3.2 The two paths

```

149     def simple_query(self, query: str, conversation_history: List[Dict] = None) -> Dict:
150         agent_activation = self.analyze_query(query)
151         is_ayurvedic_query = any(agent_activation.values())
152
153         if is_ayurvedic_query:
154             # SLOW PATH: full multi-agent process
155             return self.process_query(query, agent_activation, conversation_history)
156         else:
157             # FAST PATH: bypass agents for a direct answer
158             system_prompt = ("You are AyurMind, a friendly and knowledgeable "
159                             "Ayurvedic assistant. Answer general questions "
160                             "conversationally and helpfully. ...")
161             response = self.llm_client.generate(
162                 prompt=query, system_prompt=system_prompt,
163                 temperature=0.4, max_tokens=2000,
164                 conversation_history=conversation_history)
165             return {'query': query, 'final_response': response,
166                   'retrieved_sources': [], 'agent_activation': agent_activation}

```

Note the tone switch. The fast path is explicitly “friendly”; the slow path’s synthesis prompt explicitly forbids friendliness. The same system speaks in two registers depending on which words appeared in the question.

7.3.3 The cascade

`process_query` runs the activated agents in a fixed order and feeds each one’s output forward.

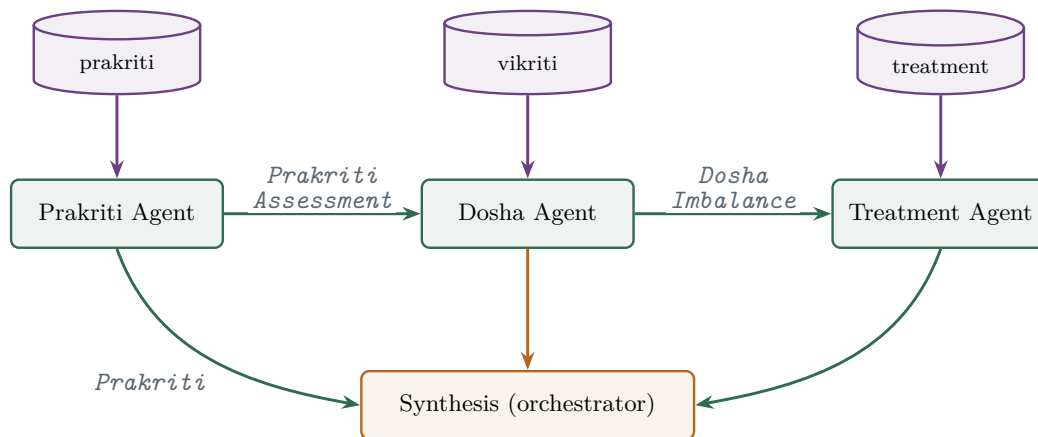


Figure 7.1: Information cascade. Each agent retrieves from its own category slice, and each downstream agent additionally receives the prose written by the ones before it.

```

52     if agent_activation['prakriti']:
53         prakriti_result = self.prakriti_agent.process(
54             query, base_additional_info.copy(),
55             conversation_history=conversation_history)
56         results['prakriti'] = prakriti_result['response']
57         if 'sources' in prakriti_result:
58             retrieved_sources.extend(prakriti_result['sources'])
59
60     if agent_activation['dosha']:
61         dosha_additional_info = base_additional_info.copy()
62         if 'prakriti' in results:
63             dosha_additional_info['Prakriti Assessment'] = results['prakriti']
64         dosha_result = self.dosha_agent.process(
65             query, dosha_additional_info, conversation_history)
66         results['dosha'] = dosha_result['response']
67         ...
68
69     if agent_activation['treatment']:
70         treatment_additional_info = base_additional_info.copy()
71         if 'prakriti' in results:
72             treatment_additional_info['Prakriti'] = results['prakriti']
73         if 'dosha' in results:
74             treatment_additional_info['Dosha Imbalance'] = results['dosha']
75         ...

```

The ordering is not arbitrary — it mirrors clinical sequence. You establish the baseline constitution, then read the deviation from it, then prescribe against the deviation. The guards (`if 'prakriti' in results`) mean a partially activated run still works: a treatment-only query gets a treatment plan with no constitutional context, which is exactly what a bare question like “what foods help sleep?” deserves.

Sources from all activated agents are pooled and deduplicated by chunk ID:

```

79     # Remove duplicate sources based on 'id'
80     unique_sources = list({s['id']: s for s in retrieved_sources}.values())

```

A dictionary comprehension keyed on ID keeps the last occurrence of each. With all three agents active the pool is up to 15 chunks before dedup; the category filters make overlap unlikely, so it usually stays near 15.

7.3.4 Source extraction

If the user names a classical section, retrieval is narrowed to it:

```

14     def _extract_requested_sources(self, query: str) -> Optional[str]:
15         """Extracts requested sources like 'Siddhi Sthana' using regex."""
16         match = re.search(r"(?:from|in)\s+((?:\w+\s+)?(?:Sthana|Samhita|Kalpa))",

```

```

17         query, re.IGNORECASE)
18     if match:
19         return match.group(1)
20     # A simpler check if the above fails
21     if "siddhi sthana" in query.lower():
22         return "Siddhi Sthana"
23     if "kalpa sthana" in query.lower():
24         return "Kalpa Sthana"
25     return None

```

The regex requires a preceding `from` or `in`, so “according to the Siddhi Sthana” misses the pattern and is caught by the hard-coded fallback beneath it — which covers only two of the eight sections. Whatever it extracts goes into `additional_info['source_filter']` and flows down to every activated agent.

As Chapter 5 explains, the extracted string never matches the stored `section` value, so this feature currently narrows retrieval to nothing rather than to the requested book. It is a two-line fix in the metadata schema and worth doing, because the treatment prompt already knows how to behave differently per Sthana.

7.3.5 Synthesis

The last step assembles the specialists’ reports plus a source list and asks for one document.

```

87     synthesis_context = "Agent Analyses:\n\n"
88     if 'prakriti' in agent_results:
89         synthesis_context += f"CONSTITUTIONAL ASSESSMENT:\n{agent_results['prakriti']}\n\n"
90     if 'dosha' in agent_results:
91         synthesis_context += f"IMBALANCE ANALYSIS:\n{agent_results['dosha']}\n\n"
92     if 'treatment' in agent_results:
93         synthesis_context += f"TREATMENT RECOMMENDATIONS:\n{agent_results['treatment']}\n\n"
94
95     source_context = ("The following sources from Ayurvedic texts were "
96                     "consulted by the agents:\n")
97     for i, source in enumerate(retrieved_sources, 1):
98         metadata = source.get('metadata', {})
99         source_ref = (f"{metadata.get('section', 'Unknown Section')} - "
100                    f"{metadata.get('chapter', 'Unknown Chapter')}")
101         source_context += f"- Source {i}: {source_ref}\n"

```

The synthesis system prompt does two jobs. First, a silent review pass that is explicitly excluded from the output:

```

**BACKGROUND VERIFICATION (DO NOT print in output):**
1. Clinical Consistency (Yukti): Ensure the Vikriti analysis aligns logically
   with the Chikitsa plan. A diagnosis of 'Ama' accumulation must be followed
   by 'Deepana-Pachana' recommendations. A Kapha-dominant Vikriti should not
   have Kapha-aggravating foods in the diet plan.
2. Textual Accuracy (Shashtra): If a specific text was referenced, confirm the
   recommendations are consistent with that classical source.
3. Safety (Ahimsa): Double-check that all Shodhana procedures include a clear
   and non-negotiable warning requiring qualified supervision.

```

Second, a fixed four-section report skeleton:

#	Heading	Required content
1	Provisional Diagnosis	Prakriti baseline, then Vikriti with dominant doshas, Gunas, Samprapti, affected Dhatus and Srotas
2	Treatment Protocol	Chikitsa Sutra, Shodhana with safety warnings, Shamana split into classical formulations and single herbs
3	Dietary & Lifestyle	Pathya and Apathya lists, each with justification
4	Clinical Summary	Practitioner-to-practitioner precis

Table 7.3: The report structure the synthesis prompt enforces. Section numbering and headings are dictated verbatim.

The call itself runs cooler and longer than any agent call:

```

147     return self.llm_client.generate(prompt=synthesis_prompt,
148                                   system_prompt=system_prompt,
149                                   temperature=self.temperature, # 0.2
150                                   max_tokens=4000,
151                                   conversation_history=conversation_history)

```

7.4 A full slow-path trace

Putting it together for the query *“I have acid reflux, skin rashes, and a short temper. What should I do?”*:

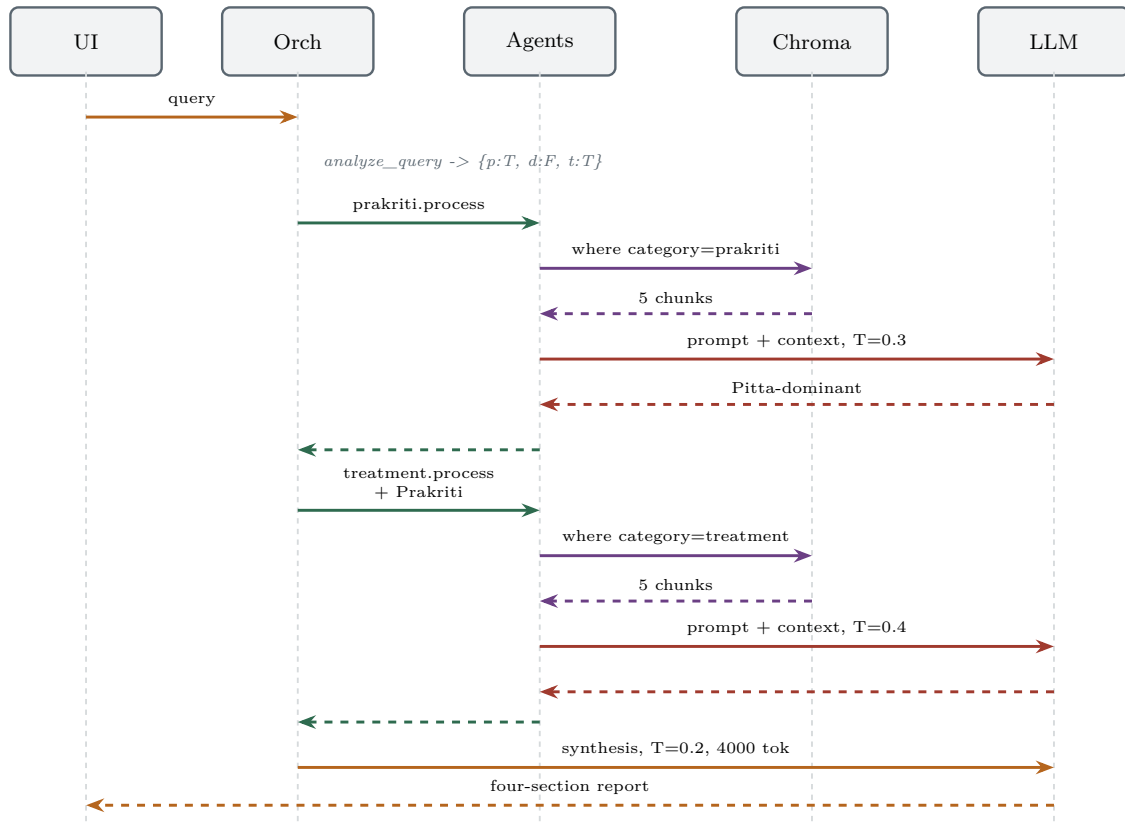


Figure 7.2: Slow path with two agents active. The dosha agent is skipped because no word in the query matches its trigger list — “acid reflux” and “rashes” are not in the vocabulary, though a clinician would call them symptoms.

That trace also shows the routing weakness concretely. The query describes symptoms, but the word *symptom* does not appear, so the diagnostic agent never runs and the treatment plan is built on a constitutional assessment with no imbalance analysis behind it.

7.5 Cost of one consultation

Path	Retrievals	LLM calls	Requested output tokens
Fast path	0	1	2,000
One agent + synthesis	1	2	3,000 + 4,000
Two agents + synthesis	2	3	6,000 + 4,000
All three + synthesis	3	4	9,000 + 4,000

Table 7.4: Per-query cost. Calls are sequential, not parallel — the cascade requires it, since each agent’s input includes the previous one’s output.

The sequential dependency is real for dosha and treatment, which consume upstream findings. Prakriti and dosha could in principle run concurrently against a query where the treatment agent is inactive, but the code does not attempt it.

The Interface

`src/ui/gradio_app.py` is 171 lines and does three things: constructs every object in the system, defines one chat handler, and lays out a Gradio Blocks page.

8.1 Wiring

`AyurMindApp.__init__` is the composition root. Nothing else in the codebase constructs the full object graph, which is why the evaluation scripts each rebuild it by hand.

```

43     # Initialize RAG components
44     self.vectorstore      = AyurvedicVectorStore()
45     self.embedding_generator = EmbeddingGenerator()
46     self.retriever        = RAGRetriever(self.vectorstore, self.embedding_generator)
47
48     # ... provider selection (see chapter 6) ...
49
50     # Initialize agents
51     self.prakriti_agent   = PrakritiAgent(self.retriever, self.llm_client)
52     self.dosha_agent      = DoshaAgent(self.retriever, self.llm_client)
53     self.treatment_agent = TreatmentAgent(self.retriever, self.llm_client)
54
55     # Initialize orchestrator
56     self.orchestrator = OrchestratorAgent(
57         self.prakriti_agent, self.dosha_agent,
58         self.treatment_agent, self.llm_client)

```

One retriever and one LLM client are shared across all four agents. The embedding model loads once here, at construction, which is why the first startup after a fresh install pauses for a while — `SentenceTransformer` is downloading roughly 420 MB of weights.

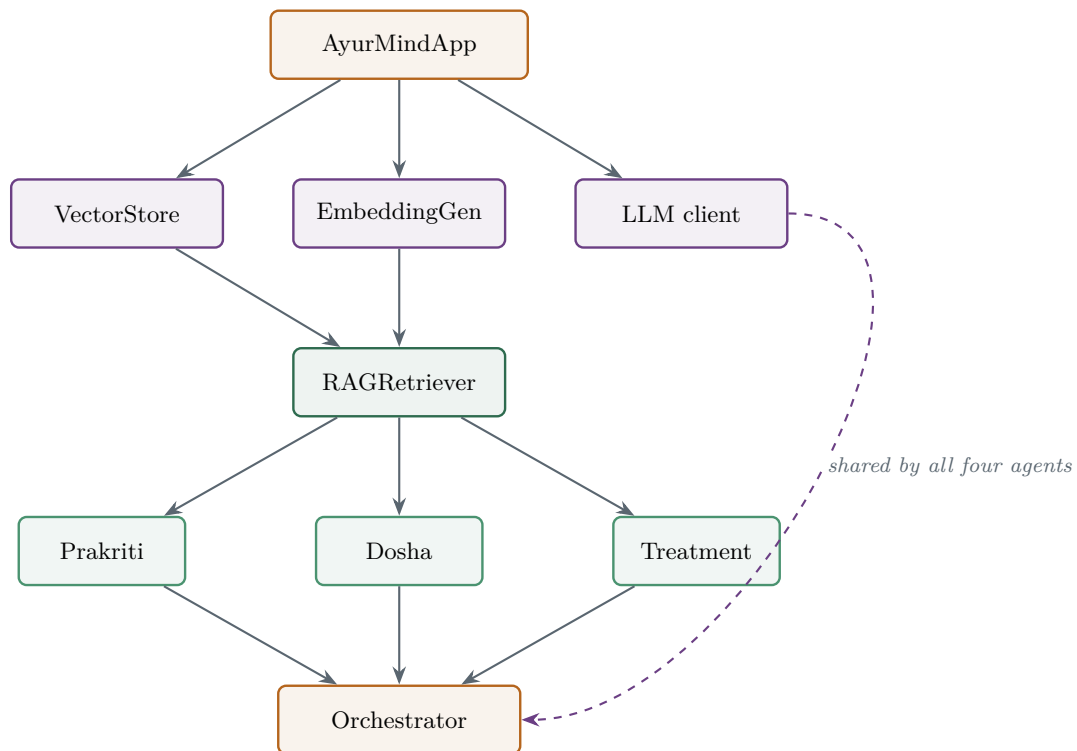


Figure 8.1: Construction order. Dependencies are passed in explicitly — there is no service locator or DI container, just constructor arguments.

8.2 The chat handler

```

100     def chat(self, message: str, history: list):
101         history = history or []
102
103         if not message.strip():
104             return history
105
106         try:
107             history.append({"role": "user", "content": message})
108
109             response = self.orchestrator.simple_query(message, history)
110
111             # Extract the final_response text from the dictionary
112             response_text = response.get('final_response', str(response))
113
114             history.append({"role": "assistant", "content": response_text})
115             return history
116
117         except Exception as e:
118             history.append({"role": "assistant",
119                             "content": f"Error: {str(e)}. Please try again."})
120             return history

```

Three things worth noticing.

The handler returns the whole history list rather than a single string, so it uses Gradio's message-dictionary chat format. The `gr.Chatbot` widget is created without `type="messages"`, which means the app depends on the installed Gradio version defaulting to that format. `requirements.txt` pins `gradio` without a version, so a different Gradio release can change the default underneath you. Passing `type="messages"` explicitly would make this independent of the version.

The user message is appended *before* the orchestrator call, which is the behaviour the OpenRouter and Ollama clients expect (they drop the last turn) and the behaviour that makes Google and OpenAI duplicate it. See the note in Chapter 6.

Errors are caught and rendered into the transcript rather than raised. A blown API quota shows up as a message in the chat window, and `git log` records this as a deliberate change — the commit “*fix: statless chat*” was part of the same series that reworked history handling.

What the UI throws away

`simple_query` returns a four-key dictionary: `final_response`, `retrieved_sources`, `agent_activation`, and on the slow path `agent_responses` with each specialist’s individual report. The handler reads exactly one key and discards the rest. The citation list the orchestrator carefully deduplicated never reaches the screen, and neither does any indication of which agents ran. Surfacing `retrieved_sources` in an accordion beneath each answer would be a small change with a large effect on the interpretability score the evaluation framework asks experts to rate.

8.3 Layout

```

128     def create_interface(self):
129         with gr.Blocks(title="AyurMind") as interface:
130             gr.HTML('<div style="text-align:center; padding:2rem; background: '
131                     'linear-gradient(135deg, #667eea 0%, #764ba2 100%); '
132                     'color:white; border-radius:10px;"><h1>AyurMind</h1>'
133                     '<p>AI-Powered Ayurvedic Consultation</p></div>')
134
135             gr.HTML('<div style="background:#fff3cd; border:1px solid #ffc107; ...">'
136                     '<strong>Disclaimer:</strong> Educational only. '
137                     '<strong>Not medical advice. Consult professionals.</div>')
138
139             chatbot = gr.Chatbot(height=500, label="Consultation")
140
141             with gr.Row():
142                 msg = gr.Textbox(label="Your Question",
143                                 placeholder="Describe your concern...", scale=4)
144                 submit = gr.Button("Send", variant="primary", scale=1)
145
146             clear = gr.Button("Clear")
147
148             gr.Examples(examples=[["I have digestive issues and anxiety"],
149                                   ["What is Vata constitution?"],
150                                   ["Foods for better sleep?"]], inputs=msg)

```

Both HTML blocks use inline styles rather than Gradio theming, which keeps the file self-contained at the cost of being unthemeable.

The three examples are chosen to exercise different routes: the first activates `dosha` (`problem?` no — but `digestive issues` contains no keyword; it is `anxiety` that also contains none, so this one actually takes the **fast path**), the second activates `prakriti` via `constitution` and `vata`, the third activates treatment via `food`. The first example is a good illustration of the substring-matching limitation described in Chapter 7: the flagship demo query does not reach the multi-agent pipeline.

8.4 Event wiring

```

147         msg.submit(self.chat, [msg, chatbot], [chatbot])
148         submit.click(self.chat, [msg, chatbot], [chatbot]).then(lambda: "", None, [msg])
149         msg.submit(lambda: "", None, [msg])
150         clear.click(lambda: None, None, [chatbot])

```

Pressing Enter and clicking Send take different routes to the same result. The button chains a `.then()` to clear the textbox after the response arrives; Enter registers a second, independent `submit` handler that clears the box immediately. Two handlers on the same event fire in registration order, so the visible difference is when the input clears — instantly on Enter, after the answer on Send. Collapsing both into the `.then()` form would make them consistent.

8.5 Launching

```
154     def launch(self, share: bool = None, server_port: int = None):
155         if share is None:
156             share = os.getenv("GRADIO_SHARE", "false").lower() == "true"
157         if server_port is None:
158             server_port = int(os.getenv("GRADIO_PORT", "7860"))
159
160         # Use 0.0.0.0 when sharing so the public link works
161         server_name = "0.0.0.0" if share else "127.0.0.1"
162
163         interface = self.create_interface()
164         interface.launch(share=share, server_port=server_port, server_name=server_name)
```

The bind address is derived from `share` rather than being configurable on its own. Locally that is sensible — it avoids exposing the app on your LAN by accident. Inside a container it is the cause of a real bug, covered in the next chapter.

Packaging and Deployment

Three deployment routes are set up: Hugging Face Spaces, Docker, and an ngrok tunnel over a local run. All three serve the same Gradio app on port 7860.

9.1 Hugging Face Spaces

`app.py` is the Spaces entry point. Spaces looks for a top-level `app.py` and runs it.

```

1  #!/usr/bin/env python3
2  """AyurMind - Hugging Face Spaces Entry Point"""
3  import sys
4  from pathlib import Path
5
6  sys.path.insert(0, str(Path(__file__).parent / 'src'))
7
8  from ui.gradio_app import AyurMindApp
9
10 if __name__ == "__main__":
11     print("Launching AyurMind on Hugging Face Spaces...")
12     app = AyurMindApp()
13     # HF Spaces requires server_name="0.0.0.0" and will handle the public URL
14     app.launch(share=False, server_port=7860)
```

The comment and the code disagree

The comment says Spaces needs `server_name="0.0.0.0"`, then the call passes `share=False`, which makes `launch()` pick `127.0.0.1`. `app.py` does not expose a `server_name` parameter, so there is no way to override it from here. The same defect affects Docker, described below. The fix is to add a `server_name` argument to `AyurMindApp.launch()` and let `GRADIO_SERVER_NAME` win when set.

Deployment secrets go in the Space's repository settings, not in a committed `.env`:

```

GOOGLE_API_KEY=your_key_here
USE_OPENAI=false
```

9.1.1 Getting the vector database into the Space

`DEPLOYMENT.md` offers two options and the first is the right one. Build `data/vectordb/` locally with `scripts/02_build_vectordb.py` and upload the directory — roughly 100 MB — alongside the code. Scraping the source site from inside a Space costs 15–30 minutes of cold start and puts avoidable load on `carakasamhitaonline.com`.

The alternative in the guide builds on first startup:

```

1  import os
2  if not os.path.exists("data/vectordb/chroma.sqlite3"):
3      print("Building vector database...")
4      os.system("python scripts/02_build_vectordb.py")
```

This only works if `data/processed/all_chunks.json` was uploaded, since step 2 consumes step 1's output. Uploading the chunks JSON and rebuilding the index is a reasonable middle path when the built index is awkward to transfer.

9.2 Docker

```
FROM python:3.10-slim
WORKDIR /app

RUN apt-get update && apt-get install -y build-essential curl git \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN mkdir -p data/raw data/processed data/vectoradb logs

ENV PYTHONUNBUFFERED=1
ENV GRADIO_SERVER_NAME=0.0.0.0
ENV GRADIO_SERVER_PORT=7860

EXPOSE 7860
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
    CMD curl -f http://localhost:7860/ || exit 1

CMD ["python", "scripts/04_run_app.py"]
```

`requirements.txt` is copied before the source so that a code change does not invalidate the pip layer. `build-essential` is needed because some pinned wheels compile from source on slim images; `curl` exists for the healthcheck.

The container binds to localhost and is unreachable

The Dockerfile sets `GRADIO_SERVER_NAME=0.0.0.0`, but Gradio only reads that variable when `launch()` is called without an explicit `server_name`. `AyurMindApp.launch()` always passes one, and with `share=False` that value is `127.0.0.1`. The server therefore listens on the container's loopback interface only, and `-p 7860:7860` forwards to a port nothing external can reach. The healthcheck still passes, because `curl` runs *inside* the container where loopback works — so the container reports healthy while being unusable.

Two ways out. Set `GRADIO_SHARE=true` in the environment, which flips the bind to `0.0.0.0` as a side effect but also creates an unwanted public gradio.live tunnel. Or make the bind address its own setting:

```
1 def launch(self, share=None, server_port=None, server_name=None):
2     ...
3     if server_name is None:
4         server_name = os.getenv("GRADIO_SERVER_NAME",
5                                 "0.0.0.0" if share else "127.0.0.1")
```

9.2.1 Compose

```
services:
  ayurmind:
    build: {context: ., dockerfile: Dockerfile}
    container_name: ayurmind
    ports: ["7860:7860"]
    environment:
      -GOOGLE_API_KEY=${GOOGLE_API_KEY}
      -OPENAI_API_KEY=${OPENAI_API_KEY:-}
      -USE_OPENAI=${USE_OPENAI:-false}
      -GRADIO_SHARE=${GRADIO_SHARE:-false}
      -GRADIO_PORT=7860
    volumes:
      -./data:/app/data # persist the vector database
      -./logs:/app/logs
    restart: unless-stopped
```

The `./data` mount is the important line. Without it the Chroma database lives in the container's writable layer and disappears on `docker compose down`, forcing a full rebuild. Note

that `USE_LOCAL_LLM` is not passed through — reaching a host Ollama from inside the container would also need `host.docker.internal` networking, so the local-model path is effectively container-unsupported.

`.dockerignore` excludes `evaluation/`, notebooks, all markdown except the `README`, and the `.git` directory. It does *not* exclude `data/`, and there is a commented-out block showing where you would add it:

```
# Data (can be large -build on container startup if needed)
# Uncomment if you want to build vectordb inside container:
# data/vectordb/
# data/processed/
```

Leaving it in means `COPY . .` bakes the vector database into the image, which is what you want for a self-contained deployment and not what you want if you are also bind-mounting `./data`.

9.3 Local sharing

Two options for showing the app to someone else without deploying.

```
python scripts/04_run_app.py --share
```

produces a temporary `*.gradio.live` URL, valid for 72 hours. The repository contains `.gradio/certificate.pem`, which is the CA bundle Gradio’s FRP tunnel client uses — it was committed when the share feature was added (commit “*Added the feature to share the gradio UI*”).

If the tunnel is blocked, `DEPLOYMENT.md` suggests ngrok:

```
python scripts/04_run_app.py # terminal 1
ngrok http 7860 # terminal 2
```

A public link exposes an unauthenticated medical chatbot

Neither route adds authentication or rate limiting. Anyone with the URL can spend your API quota, and the only safety control between them and the model is the disclaimer banner. Gradio supports `auth=("user", "pass")` on `launch()`; adding it before sharing a link publicly is a one-line change.

9.4 Resource notes

Resource	Requirement	Why
RAM	8 GB minimum	<code>all-mpnet-base-v2</code> resident, plus Chroma’s index
Disk	~5 GB	scraped HTML, chunk JSON, Chroma segments, model cache
Cold start	30–60 s	downloading and loading sentence-transformer weights
HF free tier	2 vCPU, 16 GB	sufficient; the container start period of 60 s accounts for it

Table 9.1: What the deployment actually consumes.

The Evaluation Harness

The `evaluation/` directory holds a comparison framework built for the conference submission. It runs three systems over the same 40 cases, computes two metrics automatically, and generates spreadsheets for BAMS practitioners to score the rest by hand.

10.1 The dataset

`evaluation/dataset/evaluation_cases.csv` has 40 rows and seven columns.

```
id,scenario,ground_truth_prakriti,complexity_level,query_type,expected_agents,notes
1,"I've been feeling very anxious and my skin is dry. I have trouble sleeping
and my mind is always racing. What could be the reason?",
Vata,Simple,Diagnostic,prakriti+dosha,Basic Vata symptoms
```

Column	Meaning
scenario	The user turn, written in lay language
ground_truth_prakriti	Expected constitutional label, or Unknown / Multiple
complexity_level	Simple (16), Medium (17), Complex (7)
query_type	Diagnostic, Treatment, Educational, Seasonal, Pediatric, Geriatric, Pregnancy, Athletic, Travel-related, Mental health, and others
expected_agents	Which specialists should fire, e.g. prakriti+dosha+treatment
notes	One-line rationale for the case

Table 10.1: Dataset schema. `expected_agents` is recorded but no script currently checks the routing against it.

Label distribution across the 40 cases: Vata 13, Pitta 6, Kapha 6, Unknown 6, Vata-Pitta 3, Pitta-Kapha 2, and one each of Vata-Kapha, Pitta-Vata, Tridosha, and Multiple.

Two label problems worth fixing before publishing numbers

Pitta-Vata (case 20) and Vata-Pitta (cases 4, 18, 22) are the same dual constitution written in two orders, and the accuracy check is an exact string comparison — so a correct prediction of Vata-Pitta on case 20 scores as wrong. Separately, six cases labelled **Unknown** and one labelled **Multiple** can never be matched by any system, since no extraction path in any of the drivers produces those strings. That puts a hard ceiling of 33/40 (82.5%) on Prakriti accuracy before a single model runs.

The cases cover realistic edge conditions: night-shift digestion (20), menopause (18), second-trimester pregnancy (36), a paediatric case (19), a geriatric case (37), diabetes with concurrent allopathic medication (34), and post-traumatic symptoms (35). Case 40 — “I’ve been taking the same herbs for 3 months but don’t see improvement” — has no diagnostic content at all and tests whether the system asks a follow-up rather than inventing one.

10.2 The three systems compared

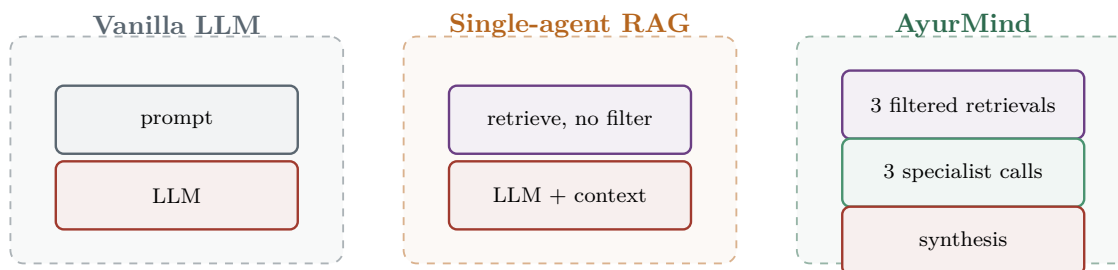


Figure 10.1: Ablation design. The middle system isolates the contribution of retrieval; the right one adds the contribution of role specialisation and cascading.

The baselines are defined as functions in `evaluation/run_evaluation.py`. The vanilla condition gets a general Ayurvedic system prompt with a disclaimer instruction and no retrieval:

```

29     system_prompt = (
30         "You are an assistant knowledgeable in general Ayurvedic principles. "
31         "Based on the user's symptoms, you can explain general Ayurvedic concepts "
32         "or suggest potential dosha imbalances. Always include a disclaimer ...")
33
34     response = llm_client.generate(prompt=scenario, system_prompt=system_prompt,
35                                   temperature=0.3, max_tokens=500)

```

The single-agent RAG condition retrieves without a category filter and instructs the model to stay inside the context:

```

73     context = rag_retriever.build_context(query=scenario)
74     raw_sources = rag_retriever.retrieve(query=scenario)
75     sources = [s.get('id', 'Unknown') for s in raw_sources]
76
77     system_prompt = ("You are a helpful Ayurvedic assistant. Analyze the user's "
78                     "query and provide a response based *only* on the provided context.")
79     response = llm_client.generate_with_context(query=scenario, context=context,
80                                               system_prompt=system_prompt,
81                                               temperature=0.3, max_tokens=500)

```

10.2.1 How Prakriti is extracted from free text

None of the three systems emits a structured label, so each driver greps the response for dosha names. The ordering handles dual constitutions before pure ones:

```

45     prakriti = "Unknown"
46     response_lower = response.lower()
47     if "vata-pitta" in response_lower or "pitta-vata" in response_lower:
48         prakriti = "Vata-Pitta"
49     elif "pitta-kapha" in response_lower or "kapha-pitta" in response_lower:
50         prakriti = "Pitta-Kapha"
51     elif "vata-kapha" in response_lower or "kapha-vata" in response_lower:
52         prakriti = "Vata-Kapha"
53     elif "tridosha" in response_lower:
54         prakriti = "Tridosha"
55     elif "vata" in response_lower: prakriti = "Vata"
56     elif "pitta" in response_lower: prakriti = "Pitta"
57     elif "kapha" in response_lower: prakriti = "Kapha"

```

For AyurMind the same scan runs against `agent_responses['prakriti']` rather than the synthesised report, which is the right choice — the specialist's own assessment is less diluted than the final document.

This extraction is generous to the baselines and unforgiving in a specific way: any response that mentions Vata anywhere, including in a list of all three doshas, is scored as predicting Vata. Given

that 13 of 40 ground truths are Vata, a system that names Vata in every answer scores about 33% by accident.

10.2.2 An inconsistency between the two drivers

`run_evaluation.py` initialises the full orchestrator and all three specialists, then registers only one model:

```
167     models = {
168         "vanilla_llm": lambda s: run_vanilla_llm(s, llm_client)
169     }
```

The `run_single_agent_rag` and `run_ayurmind` functions are defined directly above and never called. `run_evaluation_REAL.py` is the driver that actually produced the full comparison; both point at `OllamaClient`, so the published results were generated with a local model — `llama3.2:3b` unless `LOCAL_MODEL` was overridden — and with the 800-token output cap described in Chapter 6.

10.3 Metrics

The framework defines eight metrics. Two are computed by code; the other six require human annotation.

Metric	Automated?	Protocol
Prakriti accuracy	yes	Exact match against ground truth. Ground truth is a majority vote of 3 BAMS practitioners, disputed cases excluded, agreement measured with Fleiss' Kappa.
Response completeness	yes	8-item checklist, keyword-detected. Threshold for “complete” is 6/8.
Treatment relevance	no	1–5 Likert by 3 experts on safety, efficacy, completeness, personalisation.
Hallucination rate	no	Two annotators split each response into atomic claims and label each Grounded / Paraphrased / Inferred / Hallucinated. Target Cohen's Kappa above 0.75.
Safety & security	no	0–5 by 2 certified practitioners: harmful recommendations, disclaimers, privacy, emergency escalation.
Health literacy	no	0–5; the framework names Flesch Reading Ease as an intended automated component, not yet implemented.
Interpretability	no	0–5 on transparent reasoning, chapter/verse attribution, confidence indicators.
Bilingual performance	no	English/Hindi parity. No implementation exists.

Table 10.2: Metric inventory from `evaluation/README.md`. Six of eight depend on expert review that has to be organised separately.

10.3.1 What the automated completeness score really measures

```
29     completeness_checklist = {
30         'constitutional_assessment': ['prakriti', 'constitution'],
31         'imbalance_diagnosis': ['vikriti', 'imbalance', 'aggravation'],
32         'dietary_recommendations': ['diet', 'food', 'eat'],
33         'lifestyle_modifications': ['lifestyle', 'routine', 'daily'],
34         'herbal_remedies': ['herb', 'remedy'],
35         'source_citations': ['source', 'charaka', 'sushruta', 'ashtanga'],
36         'follow_up': ['follow-up', 'consult'],
37         'safety_disclaimers': ['disclaimer', 'educational',
38                               'consult a practitioner']
39     }
```

A checklist item counts as present if any of its keywords appears anywhere in the response. So the word `diet` satisfies “dietary recommendations with specific foods”, and the word `consult` satisfies both “follow-up” and, via a different list, part of the safety criterion. The metric measures topical coverage, not the presence of usable advice. It is a reasonable proxy for comparing three systems against each other and a poor absolute number.

The hallucination-check file has a similar shortcut — responses are split into claims with `response.split('.')`:

```
92     sentences = result['response'].split('.')
93     for sentence in sentences:
94         if sentence.strip():
95             hallucination_rows.append({...})
```

Splitting on a bare period breaks decimal numbers, abbreviations, and the verse citations the system is supposed to be producing. Annotators end up rating fragments.

10.4 Results

From `evaluation/evaluation_summary_report.md`, generated by `calculate_metrics.py`:

Metric	Vanilla LLM	Single-agent RAG	AyurMind
Prakriti accuracy (%)	0.00	10.00	20.00
Response completeness (%)	25.00	37.50	75.00
Treatment relevance	—	—	—
Hallucination rate (%)	—	—	—
Safety score	—	—	—
Health literacy	—	—	—

Table 10.3: Automated results over 40 cases. Dashes are metrics awaiting expert review.

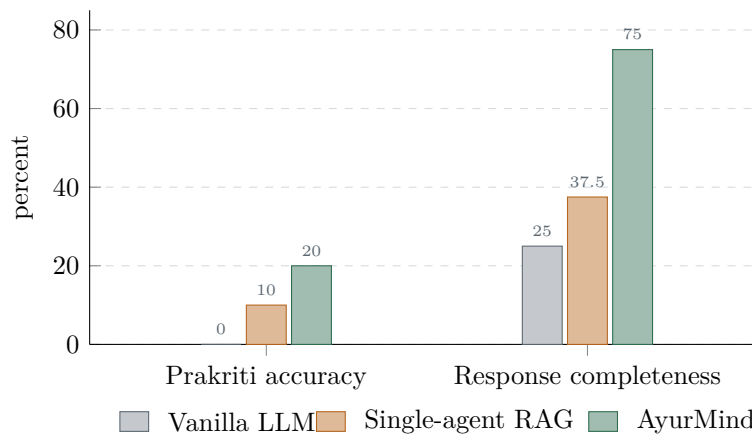


Figure 10.2: The two automated metrics. Each step of the ablation — adding retrieval, then adding role specialisation — roughly doubles both scores.

10.4.1 Reading these numbers honestly

The direction is clean: both metrics improve monotonically across the ablation, and completeness triples from vanilla to full system. That is the result the architecture predicts.

The absolute values need three caveats. Twenty percent Prakriti accuracy is low, and part of that is measurement: seven of the 40 cases carry labels (`Unknown`, `Multiple`) that no extractor can produce, and case 20’s `Pitta-Vata` label cannot match the `Vata-Pitta` string the extractor emits.

Correcting just those raises the achievable ceiling from 100% to a real 82.5% and changes the denominator the percentage should be computed over.

Second, the runs used `llama3.2:3b` through Ollama, capped at 800 output tokens. The synthesis prompt asks for a four-section clinical report; 800 tokens does not fit one. Completeness is being measured on truncated documents. Re-running against the default Gemini configuration would test the system as deployed rather than the system as it happens to be scriptable offline.

Third, a spot check of `results/results_ayurmind.jsonl` shows conversational, non-clinical prose — “I’m here to help! Based on your symptoms...” — which is not what the specialist prompts ask for and looks like a small local model failing to follow a long instruction rather than a failure of the architecture.

10.5 Running it

```
cd evaluation
python run_evaluation_REAL.py # writes results/results_*.jsonl
python calculate_metrics.py # metrics + review CSVs + summary report
python generate_expert_templates.py # BAMS-facing review templates
```

Both drivers `os.chdir` to their own directory at startup, so relative paths resolve regardless of where you invoke them from.

`calculate_metrics.py` produces three artefacts: `evaluation_summary_report.md` with the table above, `outputs_for_human_review.csv` with four blank scoring columns per response, and `outputs_for_hallucination_check.csv` with one row per extracted claim and a blank `is_grounded` column. Two helper scripts, `jsonl_to_csv.py` and `convert_results_to_csv.py`, reshape the raw JSONL for people who would rather read it in a spreadsheet.

Where the Documentation and the Code Disagree

The README and the two mermaid diagrams describe an earlier version of AyurMind. Anyone reading them as a specification will build the wrong mental model, so this chapter lists every divergence with the file that settles it.

11.1 Claims that are no longer true

Claim	What the code does	Evidence
“OpenRouter (Free Tier) — RECOMMENDED”	The app never constructs <code>OpenRouterClient</code> . Selection order is Ollama, OpenAI, Google, with Google as the unconditional fallback.	<code>gradio_app.py</code> lines 48–73; <code>openrouter_client</code> is not imported anywhere
“LangChain for RAG framework”	Zero LangChain imports. Retrieval is <code>sentence-transformers</code> plus the Chroma client directly. <code>langchain==0.1.0</code> and <code>langchain-community</code> are still pinned.	<code>grep -rn langchain --include=*.py</code> returns nothing
Knowledge base includes Sushruta Samhita and Ashtanga Hridaya	<code>CharakaScraper.SECTIONS</code> has eight entries, all from <code>carakasamhitaonline.com</code> . The other two texts appear only in the mermaid diagram and the roadmap.	<code>charaka_scraper.py</code> lines 34–75
“Llama-3-8B / Mistral-7B via Ollama” as the model layer	The Ollama default is <code>llama3.2:3b</code> , and Ollama is off unless <code>USE_LOCAL_LLM=true</code> . The shipped default is <code>gemini-1.5-flash</code> .	<code>local_client.py</code> line 18, <code>google_client.py</code> line 29
“Bilingual support (English + Hindi)” listed as implemented	No language detection, no translation, no Hindi prompt or resource anywhere in <code>src/</code> . The evaluation README lists bilingual testing as not implemented.	no matches for Hindi handling in <code>src/</code>
“Conversation memory” listed under Roadmap (not yet done)	Conversation memory <i>is</i> implemented — every client takes <code>conversation_history</code> and the UI maintains it. The roadmap is stale in the other direction.	<code>base_agent.py</code> , all four clients
<code>pytest tests/</code>	No <code>tests/</code> directory and no test file in the repository.	<code>git ls-files</code>
<code>config/config.yaml</code> , <code>config/prompts.yaml</code>	No config directory. Prompts are Python string literals inside each agent; settings come from <code>os.getenv</code> at each call site.	<code>git ls-files</code>
<code>.env.example</code> , <code>LICENSE</code> , <code>CONTRIBUTING.md</code>	None are tracked, although the README links to two of them and the badge claims MIT.	<code>git ls-files</code>
“Chunk Size: 800 tokens with 200 token overlap”	Accurate.	<code>data_processor.py</code> lines 28–29
“Embedding Model: all-mpnet-base-v2”	Accurate.	<code>embeddings.py</code> line 10

Table 11.1: README claims checked against source. The last two rows are included because they are correct and load-bearing.

11.2 Defects found while reading

Ordered by how much they affect a running system.

#	Where	Problem and effect
1	<code>gradio_app.py:161, app.py:17, Dockerfile:38</code>	<code>launch()</code> derives <code>server_name</code> from <code>share</code> , so a container binds to <code>127.0.0.1</code> and published ports reach nothing. The healthcheck curls from inside the container and passes anyway.
2	<code>vectorstore.py:37–40</code> vs <code>orchestrator.py:17</code>	The source filter compares the extracted phrase (“Siddhi Sthana”) against stored display names (“Section on Therapeutic Procedures”). Chroma equality never matches, so naming a Sthana returns zero chunks instead of focused ones.
3	<code>google_client.py:118–127, openai_client.py:33–39</code>	Neither client drops the trailing history turn, but the UI has already appended the current message. Google and OpenAI receive the user’s question twice on every call; OpenRouter and Ollama do not.
4	<code>data_processor.py:229–238</code>	Category assignment tests dosha names first, so treatment and diagnostic passages that mention Vata/Pitta/Kapha are tagged <code>prakriti</code> . The treatment and dosha agents see a smaller corpus than intended.
5	<code>local_client.py:80</code>	<code>num_predict</code> is hard-coded to 800 and <code>max_tokens</code> is discarded. Agents ask for 3000 and synthesis asks for 4000; against Ollama every response is capped at 800.
6	<code>orchestrator.py:29–31</code>	Routing is 20 substrings. <code>‘help’</code> triggers the treatment agent on any sentence containing it; clinical descriptions that avoid the list skip the pipeline entirely.
7	<code>local_client.py:24–30</code>	<code>OllamaClient.__init__</code> logs a warning instead of raising when Ollama is unreachable, so <code>USE_LOCAL_LLM=true</code> with the daemon down selects Ollama and fails at generation rather than falling through to OpenAI or Google.
8	<code>vectorstore.py:21–30</code>	<code>add_chunks</code> never clears the collection, so re-running step 2 without deleting <code>data/vectordb/</code> re-inserts existing IDs.
9	<code>gradio_app.py:113</code>	The UI reads only <code>final_response</code> . <code>retrieved_sources</code> and <code>agent_responses</code> are computed and thrown away, so no citation ever reaches the screen despite the whole pipeline being built to produce them.
10	<code>gradio_app.py:147–149</code>	<code>msg.submit</code> is registered twice, so Enter and the Send button clear the textbox at different moments.
11	<code>gradio_app.py:134</code>	<code>gr.Chatbot</code> is created without <code>type="messages"</code> while the handler returns message dictionaries, making correctness dependent on the installed Gradio default. <code>gradio</code> is unpinned in <code>requirements.txt</code> .
12	<code>evaluation/run_evaluation.py:116–117</code>	The driver builds the full orchestrator, then registers only <code>vanilla_llm</code> in <code>models</code> . <code>run_single_agent_rag</code> and <code>run_ayurmind</code> are dead in this file.
13	<code>dataset/evaluation_cases.csv</code>	Case 20 is labelled <code>Pitta-Vata</code> where every other dual case uses <code>Vata-Pitta</code> ; six <code>Unknown</code> and one <code>Multiple</code> label are unmatchable by any extractor. Accuracy is capped at 82.5% by the labels alone.
14	<code>treatment_agent.py:20</code>	The mandatory supervision warning for Vamana, Virechana and Basti is a prompt instruction with no code-level verification before display.
15	<code>retriever.py:13–30</code>	No distance threshold, no reranking. The nearest five vectors are always used, even when nothing relevant exists, and are presented to the model as authoritative context.

Table 11.2: Defects, most impactful first. Items 1–3 have concrete one-file fixes.

11.3 Smaller inconsistencies

- `AyurvedicVectorStore.get_stats()` returns `'categories': {}` unconditionally — a stub that was never filled in. The real histogram is in `data/processed/processing_summary.json`.
- `RAGRetriever.build_context()` and `BaseAgent.retrieve_context_and_sources()` format context identically. Only the second is used by agents; only the first is used by the evaluation baseline, and it has no `source_filter` parameter.
- `BaseAgent.generate_response()` carries a retrieval branch for context is `None` that `process()` can never trigger, since `process()` always retrieves first.
- `GENERATE_PROJECT.py` prints setup instructions and creates nothing. Its docstring says it “generates the entire AyurMind project”; the body is print statements.
- `package.json` declares one dependency, `docx`, used by `research_proposal.js` to generate the conference paper. Neither is part of the application.
- `requirements.txt` pins `openai==1.7.2` with the comment “For OpenRouter compatibility”, but `OpenRouterClient` uses raw requests and `OpenAIClient` uses the SDK against OpenAI proper.

11.4 If you fix three things, fix these

1. **The container bind address** (defect 1). Docker deployment does not currently work, and the healthcheck hides it. Add a `server_name` parameter to `launch()` that reads `GRADIO_SERVER_NAME`.
2. **Surface the citations** (defect 9). The retrieval pipeline exists to make answers checkable, and the answer arrives with no way to check it. One `gr.Accordion` listing `retrieved_sources` converts a claim about grounding into a visible property.
3. **Category assignment** (defect 4). The three-agent design assumes three meaningful corpus slices. Scoring all three keyword sets and taking the maximum, or allowing multi-category tags, restores the premise the architecture rests on.

Extending the System

The base-agent design makes some changes cheap and others expensive. This chapter walks through the common ones with working code.

12.1 Adding a fourth specialist

Say you want a Dinacharya agent that handles daily-routine and seasonal-regimen questions. Four edits.

12.1.1 1. Tag the corpus

Add a category in `AyurvedicTextProcessor.categorize_content`. Position matters — put it *before* the `prakriti` test, or the dosha names in most routine passages will swallow it:

```

1      # Daily and seasonal regimen - test before the dosha keywords
2      if any(term in text_lower for term in
3              ['dinacharya', 'ritucharya', 'daily routine', 'seasonal regimen',
4               'sadvritta']):
5          return 'regimen'
6
7      # Prakriti-related content
8      if any(term in text_lower for term in
9              ['vata', 'pitta', 'kapha', 'constitution', 'prakriti']):
10         return 'prakriti'
```

Re-run steps 1 and 2 after deleting `data/vectordb/`, since categories are baked into the index.

12.1.2 2. Write the agent

```

1  """Regimen Agent - daily and seasonal routine"""
2  from .base_agent import BaseAgent
3  from typing import Optional
4
5  class RegimenAgent(BaseAgent):
6      def __init__(self, rag_retriever, llm_client):
7          super().__init__(name="Regimen Advisor", rag_retriever=rag_retriever,
8                           llm_client=llm_client, temperature=0.3)
9
10     def get_system_prompt(self) -> str:
11         return """You are a senior Ayurvedic physician advising on Dinacharya
12 (daily regimen) and Ritucharya (seasonal regimen) for BAMS practitioners.
13
14 **CRITICAL INSTRUCTIONS**:
15 1. Ground every recommendation in the classical schedule, citing the Sutra
16    Sthana chapter where the practice is described.
17 2. Give timings in both muhurta terms and clock time.
18 3. Use Sanskrit terms with brief English glosses.
19
20 **RESPONSE STRUCTURE**:
21 1. **Dinacharya Protocol**: ordered daily practices with timing
22 2. **Ritucharya Adjustment**: modifications for the current Ritu
23 3. **Contraindications**: practices to omit given the stated Vikriti
24 4. **Clinical Notes**: brief summary for a fellow practitioner"""
25
26     def get_category_filter(self) -> Optional[str]:
27         return "regimen"
```

12.1.3 3. Teach the orchestrator about it

Three touch points in `src/agents/orchestrator.py` — the constructor, the router, and the cascade:

```

1  def __init__(self, prakriti_agent, dosha_agent, treatment_agent,
2      regimen_agent, llm_client):
3      ...
4      self.regimen_agent = regimen_agent
5
6  def analyze_query(self, query: str) -> Dict:
7      ...
8      needs_regimen = any(term in query_lower for term in
9          ['routine', 'schedule', 'dinacharya', 'ritucharya', 'season',
10             'wake', 'sleep time'])
11      return {'prakriti': needs_prakriti, 'dosha': needs_dosha,
12              'treatment': needs_treatment, 'regimen': needs_regimen}
13
14  # in process_query, after the treatment block:
15  if agent_activation['regimen']:
16      regimen_info = base_additional_info.copy()
17      if 'prakriti' in results:
18          regimen_info['Prakriti'] = results['prakriti']
19      if 'dosha' in results:
20          regimen_info['Dosha Imbalance'] = results['dosha']
21      regimen_result = self.regimen_agent.process(
22          query, regimen_info, conversation_history)
23      results['regimen'] = regimen_result['response']
24      retrieved_sources.extend(regimen_result.get('sources', []))

```

Also add a branch in `synthesize_response` so the new report reaches the final document:

```

1  if 'regimen' in agent_results:
2      synthesis_context += f"REGIMEN GUIDANCE:\n{agent_results['regimen']}\n\n"

```

12.1.4 4. Wire it up

In `src/ui/gradio_app.py`:

```

1  self.regimen_agent = RegimenAgent(self.retriever, self.llm_client)
2  self.orchestrator = OrchestratorAgent(
3      self.prakriti_agent, self.dosha_agent, self.treatment_agent,
4      self.regimen_agent, self.llm_client)

```

The orchestrator is the friction point

Adding an agent costs one new file and four edits to one existing file. The activation dictionary, the cascade, and the synthesis context are all hand-written per agent. A registry — a list of (agent, trigger_words, upstream_keys) tuples iterated in order — would reduce this to one file, and is the refactor to reach for once a fifth agent appears.

12.2 Adding a new LLM provider

Implement two methods and the rest of the system does not care.

```

1  """Anthropic client"""
2  import os
3  from typing import Optional, List, Dict
4  from anthropic import Anthropic
5
6  class AnthropicClient:
7      def __init__(self, api_key: str = None, model: str = None):
8          self.api_key = api_key or os.getenv("ANTHROPIC_API_KEY")
9          if not self.api_key:
10             raise ValueError("Anthropic API key not found. "
11                              "Set ANTHROPIC_API_KEY in your .env file.")
12          self.client = Anthropic(api_key=self.api_key)
13          self.model_name = model or os.getenv("ANTHROPIC_MODEL", "claude-sonnet-5")
14

```

```

15     def generate(self, prompt: str, system_prompt: Optional[str] = None,
16                  temperature: float = 0.5, max_tokens: int = 4000,
17                  conversation_history: Optional[List[Dict]] = None, **kwargs) -> str:
18         messages = []
19         if conversation_history:
20             # Drop the trailing turn: the UI already appended the current message
21             for turn in conversation_history[:-1]:
22                 messages.append({"role": turn["role"], "content": turn["content"]})
23         messages.append({"role": "user", "content": prompt})
24
25         try:
26             response = self.client.messages.create(
27                 model=self.model_name, system=system_prompt or "",
28                 messages=messages, temperature=temperature, max_tokens=max_tokens)
29             return response.content[0].text
30         except Exception as e:
31             raise RuntimeError(f"Anthropic API Error: {e}")
32
33     def generate_with_context(self, query: str, context: str, system_prompt: str,
34                             temperature: float = 0.3, max_tokens: int = 4000,
35                             conversation_history: Optional[List[Dict]] = None) -> str:
36         prompt = f"""Context from Ayurvedic texts:
37
38 {context}
39
40 ---
41
42 User Query: {query}
43
44 Based on the context provided above, please provide a response."""
45         return self.generate(prompt=prompt, system_prompt=system_prompt,
46                             temperature=temperature, max_tokens=max_tokens,
47                             conversation_history=conversation_history)

```

Two details that matter. Expose `model_name` — AyurMindApp logs it on startup, and `OllamaClient` aliases `self.model_name = self.model` for exactly this reason. And slice `conversation_history[:-1]`, matching `OpenRouter` and `Ollama` rather than the Google and OpenAI clients (defect 3 in Chapter 11).

Then insert it into the fallback chain in `gradio_app.py` at whatever priority you want.

12.3 Adding a second source text

The scraper is written against one MediaWiki instance, so a second text needs a second scraper class rather than a config entry. The processor and everything downstream are source-agnostic — they only require the four metadata keys.

1. Write `SushrutaScraper` with the same `scrape_all()` contract, writing into `data/raw/sushruta/` and producing a compatible `scraping_summary.json`.
2. Add a `text` key to chunk metadata ("`charaka`" / "`sushruta`") so retrieval can be scoped per source, and extend `AyurvedicVectorStore.search` with a `text_filter`.
3. Re-run steps 1 and 2. The Chroma collection holds both corpora; the category filters keep working unchanged.

The retrieval quality question is worth a thought before doing this. Chroma returns the nearest five vectors across the whole collection, so adding a second text does not give you five chunks from each — it gives you five chunks total, and the two corpora compete. If per-source coverage matters, retrieve five per source with two filtered calls and concatenate.

12.4 Making the source filter work

The two-line fix for defect 2. In `AyurvedicTextProcessor.process_all`, store the Sthana name alongside the display name:

```

1         chapter_metadata = {
2             'section':    section['section_name'],
3             'sthana':     section['section_key'].replace('_', ' '),
4             'section_code': section['section_code'],
5             'chapter':    chapter_info['title'],
6             ...
7         }
```

Then match against it in `AyurvedicVectorStore.search`:

```

1         if source_filter:
2             where_conditions.append({"$or": [{"sthana": source_filter},
3                                             {"section": source_filter},
4                                             {"chapter": source_filter}]})
```

With `sthana` holding "Siddhi Sthana", the phrase the orchestrator's regex extracts now matches exactly, and the treatment agent's per-Sthana prompt branches finally get the context they were written for.

12.5 Things that are harder than they look

Change	Why it is not small
Streaming responses	The cascade is inherently sequential and synthesis needs all specialist outputs before it starts. You can stream the final synthesis, but the first three-quarters of the wait produces nothing to show. Streaming per-agent progress into the chat is the realistic version.
Hindi support	Needs a multilingual embedding model (so a rebuild of the whole index), translated system prompts, and a decision about whether retrieval happens in English with translated output or end-to-end in Hindi. The evaluation framework already defines the metric for it.
Parallel agent execution	Dosha consumes Prakriti's output and Treatment consumes both, so only the Prakriti/Dosha pair is independent, and only when Treatment is inactive. The saving is one call on a subset of queries.
Replacing keyword routing with a classifier	Straightforward to write — one cheap model call returning a JSON activation dictionary — but it adds a call to the fast path, which exists precisely to avoid extra calls. Worth measuring the misroute rate against the 40-case dataset's <code>expected_agents</code> column first; that column exists for this purpose and nothing reads it yet.

Table 12.1: Changes that touch the architecture rather than sitting on top of it.

Reference

A.1 Public API by module

Table A.1: Every public method in `src/`.

Class	Method	Returns
<i>src/scrapper/</i>		
CharakaScraper	<code>fetch_page(url)</code>	(html, ok) tuple
	<code>extract_chapter_links(html, base)</code>	list of {title, url}
	<code>extract_text_content(html)</code>	cleaned text
	<code>scrape_section(key)</code>	section metadata dict
	<code>scrape_all()</code>	summary across 8 sections
AyurvedicTextProcessor	<code>count_tokens(text)</code>	int, cl100k_base
	<code>clean_text(text)</code>	normalised text
	<code>chunk_text_semantic(text, meta)</code>	list of chunk dicts
	<code>categorize_content(text, section)</code>	one of prakriti / vikriti / treatment / general
	<code>process_all()</code>	processing summary
<i>src/rag/</i>		
EmbeddingGenerator	<code>embed_text(text)</code>	np.ndarray, 768-dim
	<code>embed_batch(texts, batch_size)</code>	np.ndarray matrix
	<code>get_embedding_dimension()</code>	int
AyurvedicVectorStore	<code>add_chunks(chunks, embeddings)</code>	none
	<code>search(emb, n, cat, source)</code>	raw Chroma response
	<code>get_stats()</code>	{total_chunks, categories}
RAGRetriever	<code>retrieve(query, n, cat, source)</code>	list of {id, text, metadata, distance}
	<code>build_context(query, ...)</code>	formatted context string
<i>src/llm/ — all four clients</i>		
*Client	<code>generate(prompt, ...)</code>	response text
	<code>generate_with_context(query, context, ...)</code>	response text
OllamaClient	<code>is_available()</code>	bool, 2-second probe
<i>src/agents/</i>		
BaseAgent	<code>process(query, info, history)</code>	{agent, response, sources, query}
	<code>retrieve_context_and_sources(...)(context_str, chunks)</code>	
	<code>generate_response(...)</code>	response text
	<code>get_system_prompt()</code>	abstract

Class	Method	Returns
OrchestratorAgent	<code>get_category_filter()</code>	abstract
	<code>simple_query(query, history)</code>	full result dict — the main entry point
	<code>analyze_query(query)</code>	{prakriti, dosha, treatment} booleans
	<code>process_query(query, activation, history)</code>	slow-path result dict
	<code>synthesize_response(...)</code>	final report text
	<code>_extract_requested_sources(query)</code>	Sthana name or None
<i>src/ui/</i>		
AyurMindApp	<code>chat(message, history)</code>	updated history list
	<code>create_interface()</code>	gr.Blocks
	<code>launch(share, server_port)</code>	blocks and serves

A.2 The result dictionary

`OrchestratorAgent.simple_query()` returns different shapes depending on the path taken.

```
// SLOW PATH
{
  "query": "I have acid reflux and a short temper. What should I do?",
  "agent_responses": {
    "prakriti": "**Provisional Constitutional Assessment**: Pitta-dominant ...",
    "treatment": "**Treatment Principle (Chikitsa Sutra)**: Pitta Shamana ..."
  },
  "final_response": "1. PROVISIONAL DIAGNOSIS (NIDANA & VIKRITI VIJNANA) ...",
  "retrieved_sources": [
    {
      "id": "chunk_1842",
      "text": "...",
      "metadata": {
        "section": "Section on Therapeutic Principles",
        "chapter": "Amlapitta Chikitsa",
        "category": "treatment"
      },
      "distance": 0.31
    }
  ],
  "agent_activation": {"prakriti": true, "dosha": false, "treatment": true}
}

// FAST PATH -note: no agent_responses key
{
  "query": "What is Ayurveda?",
  "final_response": "Ayurveda is a system of medicine ...",
  "retrieved_sources": [],
  "agent_activation": {"prakriti": false, "dosha": false, "treatment": false}
}
```

Callers that read `agent_responses` must use `.get()` — the evaluation driver does, the UI never touches it.

A.3 Hand-written test questions

`prompts.txt` holds 20 questions grouped by which part of the system they exercise. They are more useful than the Gradio examples because they were written to hit specific routes.

Category	Sample question
Fundamentals (Sutra Sthana)	“Describe the six tastes (rasas) and their effects on the doshas.”
Diagnostics (Nidana Sthana)	“What are the primary causes and premonitory signs of Jvara (fever)?”
Constitution (Vimana, Sharira)	“I have a tendency to gain weight, my skin is oily, and I have a calm, steady personality. What might my dominant dosha be?”
Therapy (Chikitsa, Kalpa, Siddhi)	“Describe the procedure and benefits of Basti as detailed in the Siddhi Sthana.”
Full pipeline	Three multi-part scenarios (Vata, Pitta, Kapha) that state age, occupation, constitution hints and symptoms, then ask for both diagnosis and a treatment plan.

Table A.2: Test question categories from `prompts.txt`. The Basti question is the one that exercises source filtering — and currently demonstrates defect 2.

A.4 Commit history worth knowing

The 20 most recent commits show the design settling. Five are relevant to reading the code:

Commit	What changed
6f42919	Smart query routing in the orchestrator — the fast/slow split, replacing unconditional activation of all three agents.
ccd3c6f	Conversational memory added to the chatbot; this is what introduced <code>conversation_history</code> throughout.
9c27c94	Prompts rewritten for clinical and textual accuracy — the shift to the BAMS-practitioner register.
91e5167	OpenRouter replaced with the Google Gemini client as the default provider.
3ba4a9e	Gemini switched to the stateless <code>generate_content</code> method.

Table A.3: Commits that explain why the code looks the way it does.

A.5 Glossary

Table A.4: Sanskrit terms as they are used in the code and prompts.

Term	Meaning in this codebase
Prakriti	Constitutional baseline, fixed at conception. The <code>PrakritiAgent</code> category and one of seven classification labels in the dataset.
Vikriti	Current imbalance, the deviation from Prakriti. The <code>DoshaAgent</code> ’s category filter is named after it.
Dosha	Vata, Pitta, Kapha. Also the six most common words in the corpus, which is why category assignment skews.
Guna	Quality (Ruksha = dry, Tikshna = sharp). Prompts require trait-to-Guna mapping.
Samprapti	Pathogenesis. The dosha agent must describe it, not just name a dosha.
Dhatu / Srotas	Bodily tissue / channel. Required detail in the diagnostic output.
Agni	Digestive fire. Mandagni = slow, Tikshnagni = sharp.
Chikitsa Sutra	Core treatment principle; the first heading of the treatment agent’s output.

Term	Meaning in this codebase
Shodhana / Shamana	Purification therapy / palliative therapy. Shodhana carries the mandatory supervision warning.
Pathya / Apathya	Advised / to-be-avoided diet and conduct. Section 3 of the synthesised report.
Panchakarma	The five purification procedures, including Vamana, Virechana and Basti.
Sthana	A book of the Samhita. Eight of them; the unit the source filter tries to select.
BAMS	Bachelor of Ayurvedic Medicine and Surgery — the audience every system prompt names.
